

Table of Contents

1. Minimum Stack / Minimum Queue
2. Sparse Table
3. Two Pointers
4. Disjoint Set Union
5. Fenwick Tree
6. Sqrt Decomposition
7. Segment Tree
8. Treap
9. Sqrt Tree
10. Randomized Heap
11. Deleting from a data structure in $O(T(n) \log n)$

Translated From: e-maxx.ru

Minimum stack / Minimum queue

In this article we will consider three problems: first we will modify a stack in a way that allows us to find the smallest element of the stack in $O(1)$, then we will do the same thing with a queue, and finally we will use these data structures to find the minimum in all subarrays of a fixed length in an array in $O(n)$

Stack modification

We want to modify the stack data structure in such a way, that it is possible to find the smallest element in the stack in $O(1)$ time, while maintaining the same asymptotic behavior for adding and removing elements from the stack. Quick reminder, on a stack we only add and remove elements on one end.

To do this, we will not only store the elements in the stack, but we will store them in pairs: the element itself and the minimum in the stack starting from this element and below.

```
stack<pair<int, int>> st;
```

It is clear that finding the minimum in the whole stack consists only of looking at the value `stack.top().second`.

It is also obvious that adding or removing a new element to the stack can be done in constant time.

Implementation:

- Adding an element:

```
int new_min = st.empty() ? new_elem : min(new_elem, st.top().second);  
st.push({new_elem, new_min});
```

- Removing an element:

```
int removed_element = st.top().first;  
st.pop();
```

- Finding the minimum:

```
int minimum = st.top().second;
```

Queue modification (method 1)

Now we want to achieve the same operations with a queue, i.e. we want to add elements at the end and remove them from the front.

Here we consider a simple method for modifying a queue. It has a big disadvantage though, because the modified queue will actually not store all elements.

The key idea is to only store the items in the queue that are needed to determine the minimum. Namely we will keep the queue in nondecreasing order (i.e. the smallest value will be stored in the head), and of course not in any arbitrary way, the actual minimum has to be always contained in the queue. This way the smallest element will always be in the head of the queue. Before adding a new element to the queue, it is enough to make a "cut": we will remove all trailing elements of the queue that are larger than the new element, and afterwards add the new element to the queue. This way we don't break the order of the queue, and we will also not lose the current element if it is at any subsequent step the minimum. All the elements that we removed can never be a minimum itself, so this operation is allowed. When we want to extract an element from the head, it actually might not be there (because we removed it previously while adding a smaller element). Therefore when deleting an

element from a queue we need to know the value of the element. If the head of the queue has the same value, we can safely remove it, otherwise we do nothing.

Consider the implementations of the above operations:

```
deque<int> q;
```

- Finding the minimum:

```
int minimum = q.front();
```

- Adding an element:

```
while (!q.empty() && q.back() > new_element)
    q.pop_back();
q.push_back(new_element);
```

- Removing an element:

```
if (!q.empty() && q.front() == remove_element)
    q.pop_front();
```

It is clear that on average all these operation only take $O(1)$ time (because every element can only be pushed and popped once).

Queue modification (method 2)

This is a modification of method 1. We want to be able to remove elements without knowing which element we have to remove. We can accomplish that by storing the index for each element in the queue. And we also remember how many elements we already have added and removed.

```
deque<pair<int, int>> q;
int cnt_added = 0;
int cnt_removed = 0;
```

- Finding the minimum:

```
int minimum = q.front().first;
```

- Adding an element:

```
while (!q.empty() && q.back().first > new_element)
    q.pop_back();
q.push_back({new_element, cnt_added});
cnt_added++;
```

- Removing an element:

```
if (!q.empty() && q.front().second == cnt_removed)
    q.pop_front();
cnt_removed++;
```

Queue modification (method 3)

Here we consider another way of modifying a queue to find the minimum in $O(1)$. This way is somewhat more complicated to implement, but this time we actually store all elements. And we also can remove an element from the front without knowing its value.

The idea is to reduce the problem to the problem of stacks, which was already solved by us. So we only need to learn how to simulate a queue using two stacks.

We make two stacks, `s1` and `s2`. Of course these stack will be of the modified form, so that we can find the minimum in $O(1)$. We will add new elements to the stack `s1`, and remove elements from the stack `s2`. If at any time the stack `s2` is empty, we move all elements from `s1` to `s2` (which essentially reverses the order of those elements). Finally finding the minimum in a queue involves just finding the minimum of both stacks.

Thus we perform all operations in $O(1)$ on average (each element will be once added to stack `s1`, once transferred to `s2`, and once popped from `s2`)

Implementation:

```
stack<pair<int, int>> s1, s2;
```

- Finding the minimum:

```
if (s1.empty() || s2.empty())
    minimum = s1.empty() ? s2.top().second : s1.top().second;
else
    minimum = min(s1.top().second, s2.top().second);
```

- Add element:

```
int minimum = s1.empty() ? new_element : min(new_element, s1.top().second);
s1.push({new_element, minimum});
```

- Removing an element:

```
if (s2.empty()) {
    while (!s1.empty()) {
        int element = s1.top().first;
        s1.pop();
        int minimum = s2.empty() ? element : min(element, s2.top().second);
        s2.push({element, minimum});
    }
}
int remove_element = s2.top().first;
s2.pop();
```

Finding the minimum for all subarrays of fixed length

Suppose we are given an array `A` of length `N` and a given $M \leq N$. We have to find the minimum of each subarray of length `M` in this array, i.e. we have to find:

$$\min_{0 \leq i \leq M-1} A[i], \min_{1 \leq i \leq M} A[i], \min_{2 \leq i \leq M+1} A[i], \dots, \min_{N-M \leq i \leq N-1} A[i]$$

We have to solve this problem in linear time, i.e. $O(n)$.

We can use any of the three modified queues to solve the problem. The solutions should be clear: we add the first `M` element of the array, find and output its minimum, then add the next element to the queue and remove the first element of the array, find and output its minimum, etc.

Since all operations with the queue are performed in constant time on average, the complexity of the whole algorithm will be $O(n)$.

Practice Problems

- [Queries with Fixed Length](#)
- [Sliding Window Minimum](#)
- [Binary Land](#)

Contributors:

Jakob Kogler (92.19%) Oleksandr Kulkov (3.12%) Udaya Bhaskar (1.56%) Prashant (1.04%) Deepank Tyagi (0.52%) Saumya (0.52%)
Tanmay Sinha (0.52%) Tushar Bisht (0.52%)

Original

Sparse Table

Sparse Table is a data structure, that allows answering range queries. It can answer most range queries in $O(\log n)$, but its true power is answering range minimum queries (or equivalent range maximum queries). For those queries it can compute the answer in $O(1)$ time.

The only drawback of this data structure is, that it can only be used on *immutable* arrays. This means, that the array cannot be changed between two queries. If any element in the array changes, the complete data structure has to be recomputed.

Intuition

Any non-negative number can be uniquely represented as a sum of decreasing powers of two. This is just a variant of the binary representation of a number. E.g. $13 = (1101)_2 = 8 + 4 + 1$. For a number x there can be at most $\lceil \log_2 x \rceil$ summands.

By the same reasoning any interval can be uniquely represented as a union of intervals with lengths that are decreasing powers of two. E.g. $[2, 14] = [2, 9] \cup [10, 13] \cup [14, 14]$, where the complete interval has length 13, and the individual intervals have the lengths 8, 4 and 1 respectively. And also here the union consists of at most $\lceil \log_2(\text{length of interval}) \rceil$ many intervals.

The main idea behind Sparse Tables is to precompute all answers for range queries with power of two length. Afterwards a different range query can be answered by splitting the range into ranges with power of two lengths, looking up the precomputed answers, and combining them to receive a complete answer.

Precomputation

We will use a 2-dimensional array for storing the answers to the precomputed queries. $st[i][j]$ will store the answer for the range $[j, j + 2^i - 1]$ of length 2^i . The size of the 2-dimensional array will be $(K + 1) \times \text{MAXN}$, where MAXN is the biggest possible array length. K has to satisfy $K \geq \lceil \log_2 \text{MAXN} \rceil$, because $2^{\lceil \log_2 \text{MAXN} \rceil}$ is the biggest power of two range, that we have to support. For arrays with reasonable length ($\leq 10^7$ elements), $K = 25$ is a good value.

The MAXN dimension is second to allow (cache friendly) consecutive memory accesses.

```
int st[K + 1][MAXN];
```

Because the range $[j, j + 2^i - 1]$ of length 2^i splits nicely into the ranges $[j, j + 2^{i-1} - 1]$ and $[j + 2^{i-1}, j + 2^i - 1]$, both of length 2^{i-1} , we can generate the table efficiently using dynamic programming:

```
std::copy(array.begin(), array.end(), st[0]);

for (int i = 1; i <= K; i++)
    for (int j = 0; j + (1 << i) <= N; j++)
        st[i][j] = f(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
```

The function f will depend on the type of query. For range sum queries it will compute the sum, for range minimum queries it will compute the minimum.

The time complexity of the precomputation is $O(N \log N)$.

Range Sum Queries

For this type of queries, we want to find the sum of all values in a range. Therefore the natural definition of the function f is $f(x, y) = x + y$. We can construct the data structure with:

```
long long st[K + 1][MAXN];

std::copy(array.begin(), array.end(), st[0]);
```

```
for (int i = 1; i <= K; i++)
    for (int j = 0; j + (1 << i) <= N; j++)
        st[i][j] = st[i - 1][j] + st[i - 1][j + (1 << (i - 1))];
```

To answer the sum query for the range $[L, R]$, we iterate over all powers of two, starting from the biggest one. As soon as a power of two 2^i is smaller or equal to the length of the range ($= R - L + 1$), we process the first part of range $[L, L + 2^i - 1]$, and continue with the remaining range $[L + 2^i, R]$.

```
long long sum = 0;
for (int i = K; i >= 0; i--) {
    if ((1 << i) <= R - L + 1) {
        sum += st[i][L];
        L += 1 << i;
    }
}
```

Time complexity for a Range Sum Query is $O(K) = O(\log \text{MAXN})$.

Range Minimum Queries (RMQ)

These are the queries where the Sparse Table shines. When computing the minimum of a range, it doesn't matter if we process a value in the range once or twice. Therefore instead of splitting a range into multiple ranges, we can also split the range into only two overlapping ranges with power of two length. E.g. we can split the range $[1, 6]$ into the ranges $[1, 4]$ and $[3, 6]$. The range minimum of $[1, 6]$ is clearly the same as the minimum of the range minimum of $[1, 4]$ and the range minimum of $[3, 6]$. So we can compute the minimum of the range $[L, R]$ with:

$$\min(\text{st}[i][L], \text{st}[i][R - 2^i + 1]) \quad \text{where } i = \log_2(R - L + 1)$$

This requires that we are able to compute $\log_2(R - L + 1)$ fast. You can accomplish that by precomputing all logarithms:

```
int lg[MAXN+1];
lg[1] = 0;
for (int i = 2; i <= MAXN; i++)
    lg[i] = lg[i/2] + 1;
```

Alternatively, log can be computed on the fly in constant space and time:

```
// C++20
#include <bit>
int log2_floor(unsigned long i) {
    return std::bit_width(i) - 1;
}

// pre C++20
int log2_floor(unsigned long long i) {
    return i ? __builtin_clzll(1) - __builtin_clzll(i) : -1;
}
```

This benchmark shows that using `lg` array is slower because of cache misses.

Afterwards we need to precompute the Sparse Table structure. This time we define f with $f(x, y) = \min(x, y)$.

```
int st[K + 1][MAXN];

std::copy(array.begin(), array.end(), st[0]);

for (int i = 1; i <= K; i++)
    for (int j = 0; j + (1 << i) <= N; j++)
        st[i][j] = min(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
```

And the minimum of a range $[L, R]$ can be computed with:

```
int i = lg[R - L + 1];
int minimum = min(st[i][L], st[i][R - (1 << i) + 1]);
```

Time complexity for a Range Minimum Query is $O(1)$.

Similar data structures supporting more types of queries

One of the main weakness of the $O(1)$ approach discussed in the previous section is, that this approach only supports queries of [idempotent functions](#). I.e. it works great for range minimum queries, but it is not possible to answer range sum queries using this approach.

There are similar data structures that can handle any type of associative functions and answer range queries in $O(1)$. One of them is called [Disjoint Sparse Table](#). Another one would be the [Sqrt Tree](#).

Practice Problems

- [SPOJ - RMQSQ](#)
- [SPOJ - THRBL](#)
- [Codechef - MSTICK](#)
- [Codechef - SEAD](#)
- [Codeforces - CGCDSSQ](#)
- [Codeforces - R2D2 and Droid Army](#)
- [Codeforces - Maximum of Maximums of Minimums](#)
- [SPOJ - Miraculous](#)
- [DevSkill - Multiplication Interval \(archived\)](#)
- [Codeforces - Animals and Puzzles](#)
- [Codeforces - Trains and Statistics](#)
- [SPOJ - Postering](#)
- [SPOJ - Negative Score](#)
- [SPOJ - A Famous City](#)
- [SPOJ - Diferencija](#)
- [Codeforces - Turn off the TV](#)
- [Codeforces - Map](#)
- [Codeforces - Awards for Contestants](#)
- [Codeforces - Longest Regular Bracket Sequence](#)
- [Codeforces - Array Stabilization \(GCD version\)](#)

Contributors:

[Jakob Kogler](#) (55.68%) [Luke Videckis](#) (15.91%) [Madhur Chauhan](#) (7.39%) [Morass](#) (7.39%) [wilwxk](#) (3.98%) [Oleksandr Kulkov](#) (3.41%)
[Anton Älgmyr](#) (2.27%) [Vidit Jain](#) (1.7%) [Ajay Maheshwari](#) (0.57%) [Gvne](#) (0.57%) [SiddharthEEE](#) (0.57%) [wiktorkuchta](#) (0.57%)

Two Pointer Algorithm

Overview

- Note: Added my me.

The Two Pointer Algorithm is a technique used for efficiently solving problems with linear data structures like arrays or linked lists. It involves using two pointers to traverse the data structure and perform operations based on certain conditions.

Basic Idea

The basic idea behind the Two Pointer Algorithm is to maintain two pointers that traverse the data structure in a way that reduces the time complexity. The pointers move towards each other or in a specific pattern based on the problem requirements.

Algorithm Steps

1. **Initialize Pointers:**
2. Initialize two pointers, usually at the beginning or end of the data structure.
3. **Move Pointers:**
4. Move the pointers towards each other, closer or farther based on the problem requirements.
5. Update pointers based on specific conditions.
6. **Check Conditions:**
7. Check conditions at each step to determine if the solution is found or if the pointers need to be adjusted.
8. **Repeat:**
9. Repeat steps 2 and 3 until the pointers meet or fulfill the problem requirements.

Common Use Cases

1. **Array Sum/Pair Sum:**
2. Finding pairs in a sorted/unordered array that sum up to a given target.
3. **Search in a Sorted Array:**
4. Searching for a target element efficiently in a sorted array.
5. **Palindrome Check:**
6. Checking if a given string or array is a palindrome.
7. **Trapping Rain Water:**
8. Calculating the amount of water that can be trapped between bars in an elevation map.

Example

Two Sum Problem

```
int main() {  
    // Example input vector  
    std::vector<int> nums = {2, 7, 11, 15};  
    int target = 9;  
    // Sorting the input vector  
    std::sort(nums.begin(), nums.end());
```

```

// Initializing two pointers
int left = 0;
int right = nums.size() - 1;
// Two Pointer Algorithm
while (left < right) {
    int currentSum = nums[left] + nums[right];
    if (currentSum == target) {
        // Solution found
        std::cout << "Pair found: [" << nums[left] << ", " << nums[right] << "]" << std::endl;
        return 0;
    } else if (currentSum < target) {
        // Move left pointer to the right
        ++left;
    } else {
        // Move right pointer to the left
        --right;
    }
}
// No solution found
std::cout << "No pair found." << std::endl;
return 0;
}

```

```

#include <cassert>
#include <iostream>
#include <vector>

/**
 * @brief Calculates the maximum water that can be trapped between vertical
 * lines on a 2D plane.
 *
 * Given an array of non-negative integers representing vertical lines on a 2D
 * plane, where the width of each line is 1, the function finds two lines that,
 * together with the x-axis, form a container that can hold the most water.
 *
 * The problem can be efficiently solved using a two-pointer approach. We
 * initialize two pointers, 'left' and 'right', at the beginning and end of the
 * array, respectively. The width of the container is determined by the
 * difference between the 'right' and 'left' pointers. To maximize the water
 * trapped, we choose the minimum height between the lines at the 'left' and
 * 'right' positions. We then calculate the water trapped by multiplying the
 * width and height. We move the pointers towards each other, adjusting them
 * based on the shorter line, as moving the pointer with the smaller height has
 * the potential to increase the water volume.
 *
 * @param height Vector representing the heights of vertical lines.
 * @return int Maximum water that can be trapped.
 */
int maxArea(const std::vector<int>& height) {
    int maxWater = 0;
    int left = 0;
    int right = height.size() - 1;

    while (left < right) {
        int width = right - left;
        int h = std::min(height[left], height[right]);
        int currentWater = width * h;
        maxWater = std::max(maxWater, currentWater);

        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }

    return maxWater;
}

/**
 * @brief Tests the maxArea function with various test cases.
 *
 * The function iterates through each test case, calculates the result using the
 * maxArea function, checks if the result matches the expected result, and
 * prints a success message for each test case.
 */

```

```
*/  
  
void testContainerWithMostWater() {  
    // Test cases with expected maximum water values  
    std::vector<std::pair<std::vector<int>, int>> testCases = {  
        { {1, 8, 6, 2, 5, 4, 8, 3, 7}, 49 },  
        { {1, 1}, 1 },  
        { {4, 3, 2, 1, 4}, 16 },  
        { {1, 2, 1}, 2 },  
    };  
    for (const auto& testCase : testCases) {  
        const std::vector<int>& input = testCase.first;  
        int expected = testCase.second;  
  
        // Calculate the result using the maxArea function  
        int result = maxArea(input);  
        assert(result == expected);  
        std::cout << "Test passed!" << std::endl;  
    }  
}  
  
int main() {  
    testContainerWithMostWater();  
    return 0;  
}
```

Contributors:

Your name (100.0%)

Translated From: e-maxx.ru

Disjoint Set Union

This article discusses the data structure **Disjoint Set Union** or **DSU**. Often it is also called **Union Find** because of its two main operations.

This data structure provides the following capabilities. We are given several elements, each of which is a separate set. A DSU will have an operation to combine any two sets, and it will be able to tell in which set a specific element is. The classical version also introduces a third operation, it can create a set from a new element.

Thus the basic interface of this data structure consists of only three operations:

- `make_set(v)` - creates a new set consisting of the new element `v`
- `union_sets(a, b)` - merges the two specified sets (the set in which the element `a` is located, and the set in which the element `b` is located)
- `find_set(v)` - returns the representative (also called leader) of the set that contains the element `v`. This representative is an element of its corresponding set. It is selected in each set by the data structure itself (and can change over time, namely after `union_sets` calls). This representative can be used to check if two elements are part of the same set or not. `a` and `b` are exactly in the same set, if `find_set(a) == find_set(b)`. Otherwise they are in different sets.

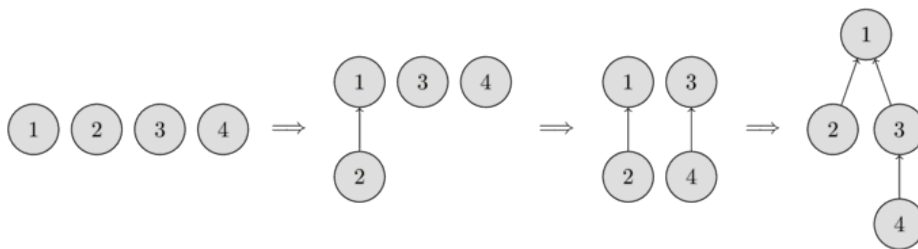
As described in more detail later, the data structure allows you to do each of these operations in almost $O(1)$ time on average.

Also in one of the subsections an alternative structure of a DSU is explained, which achieves a slower average complexity of $O(\log n)$, but can be more powerful than the regular DSU structure.

Build an efficient data structure

We will store the sets in the form of **trees**: each tree will correspond to one set. And the root of the tree will be the representative/leader of the set.

In the following image you can see the representation of such trees.



In the beginning, every element starts as a single set, therefore each vertex is its own tree. Then we combine the set containing the element 1 and the set containing the element 2. Then we combine the set containing the element 3 and the set containing the element 4. And in the last step, we combine the set containing the element 1 and the set containing the element 3.

For the implementation this means that we will have to maintain an array `parent` that stores a reference to its immediate ancestor in the tree.

Naive implementation

We can already write the first implementation of the Disjoint Set Union data structure. It will be pretty inefficient at first, but later we can improve it using two optimizations, so that it will take nearly constant time for each function call.

As we said, all the information about the sets of elements will be kept in an array `parent`.

To create a new set (operation `make_set(v)`), we simply create a tree with root in the vertex `v`, meaning that it is its own ancestor.

To combine two sets (operation `union_sets(a, b)`), we first find the representative of the set in which `a` is located, and the representative of the set in which `b` is located. If the representatives are identical, that we have nothing to do, the sets are already merged. Otherwise, we can simply specify that one of the representatives is the parent of the other representative - thereby combining the two trees.

Finally the implementation of the find representative function (operation `find_set(v)`): we simply climb the ancestors of the vertex `v` until we reach the root, i.e. a vertex such that the reference to the ancestor leads to itself. This operation is easily implemented recursively.

```
void make_set(int v) {
    parent[v] = v;
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b)
        parent[b] = a;
}
```

However this implementation is inefficient. It is easy to construct an example, so that the trees degenerate into long chains. In that case each call `find_set(v)` can take $O(n)$ time.

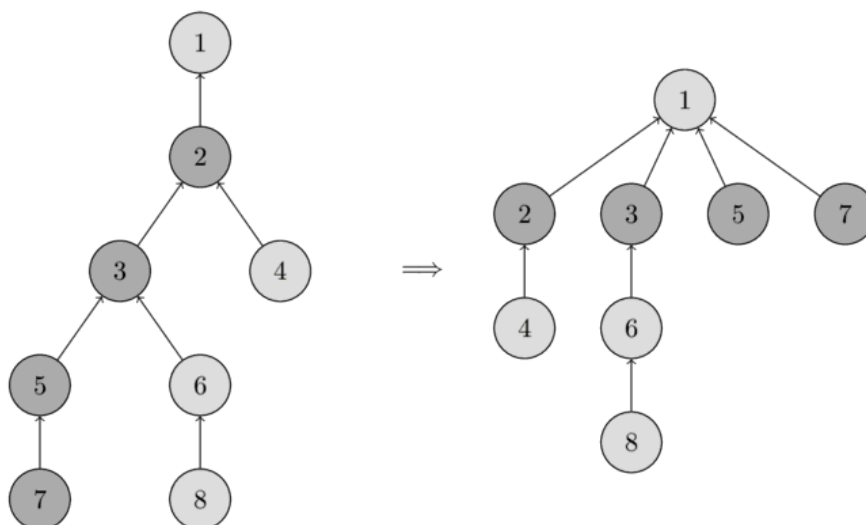
This is far away from the complexity that we want to have (nearly constant time). Therefore we will consider two optimizations that will allow to significantly accelerate the work.

Path compression optimization

This optimization is designed for speeding up `find_set`.

If we call `find_set(v)` for some vertex `v`, we actually find the representative `p` for all vertices that we visit on the path between `v` and the actual representative `p`. The trick is to make the paths for all those nodes shorter, by setting the parent of each visited vertex directly to `p`.

You can see the operation in the following image. On the left there is a tree, and on the right side there is the compressed tree after calling `find_set(7)`, which shortens the paths for the visited nodes 7, 5, 3 and 2.



The new implementation of `find_set` is as follows:

```
int find_set(int v) {
    if (v == parent[v])
        return v;
    parent[v] = find_set(parent[v]);
    return parent[v];
}
```

```

        return v;
    return parent[v] = find_set(parent[v]);
}

```

The simple implementation does what was intended: first find the representative of the set (root vertex), and then in the process of stack unwinding the visited nodes are attached directly to the representative.

This simple modification of the operation already achieves the time complexity $O(\log n)$ per call on average (here without proof). There is a second modification, that will make it even faster.

Union by size / rank

In this optimization we will change the `union_set` operation. To be precise, we will change which tree gets attached to the other one. In the naive implementation the second tree always got attached to the first one. In practice that can lead to trees containing chains of length $O(n)$. With this optimization we will avoid this by choosing very carefully which tree gets attached.

There are many possible heuristics that can be used. Most popular are the following two approaches: In the first approach we use the size of the trees as rank, and in the second one we use the depth of the tree (more precisely, the upper bound on the tree depth, because the depth will get smaller when applying path compression).

In both approaches the essence of the optimization is the same: we attach the tree with the lower rank to the one with the bigger rank.

Here is the implementation of union by size:

```

void make_set(int v) {
    parent[v] = v;
    size[v] = 1;
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (size[a] < size[b])
            swap(a, b);
        parent[b] = a;
        size[a] += size[b];
    }
}

```

And here is the implementation of union by rank based on the depth of the trees:

```

void make_set(int v) {
    parent[v] = v;
    rank[v] = 0;
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (rank[a] < rank[b])
            swap(a, b);
        parent[b] = a;
        if (rank[a] == rank[b])
            rank[a]++;
    }
}

```

Both optimizations are equivalent in terms of time and space complexity. So in practice you can use any of them.

Time complexity

As mentioned before, if we combine both optimizations - path compression with union by size / rank - we will reach nearly constant time queries. It turns out, that the final amortized time complexity is $O(\alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function, which grows very slowly. In fact it grows so slowly, that it doesn't exceed 4 for all reasonable n (approximately $n < 10^{600}$).

Amortized complexity is the total time per operation, evaluated over a sequence of multiple operations. The idea is to guarantee the total time of the entire sequence, while allowing single operations to be much slower than the amortized time. E.g. in our case a single call might take $O(\log n)$ in the worst case, but if we do m such calls back to back we will end up with an average time of $O(\alpha(n))$.

We will also not present a proof for this time complexity, since it is quite long and complicated.

Also, it's worth mentioning that DSU with union by size / rank, but without path compression works in $O(\log n)$ time per query.

Linking by index / coin-flip linking

Both union by rank and union by size require that you store additional data for each set, and maintain these values during each union operation. There exist also a randomized algorithm, that simplifies the union operation a little bit: linking by index.

We assign each set a random value called the index, and we attach the set with the smaller index to the one with the larger one. It is likely that a bigger set will have a bigger index than the smaller set, therefore this operation is closely related to union by size. In fact it can be proven, that this operation has the same time complexity as union by size. However in practice it is slightly slower than union by size.

You can find a proof of the complexity and even more union techniques [here](#).

```
void make_set(int v) {
    parent[v] = v;
    index[v] = rand();
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (index[a] < index[b])
            swap(a, b);
        parent[b] = a;
    }
}
```

It's a common misconception that just flipping a coin, to decide which set we attach to the other, has the same complexity. However that's not true. The paper linked above conjectures that coin-flip linking combined with path compression has complexity $\Omega\left(n \frac{\log n}{\log \log n}\right)$. And in benchmarks it performs a lot worse than union by size/rank or linking by index.

```
void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (rand() % 2)
            swap(a, b);
        parent[b] = a;
    }
}
```

Applications and various improvements

In this section we consider several applications of the data structure, both the trivial uses and some improvements to the data structure.

Connected components in a graph

This is one of the obvious applications of DSU.

Formally the problem is defined in the following way: Initially we have an empty graph. We have to add vertices and undirected edges, and answer queries of the form (a, b) - "are the vertices a and b in the same connected component of the graph?"

Here we can directly apply the data structure, and get a solution that handles an addition of a vertex or an edge and a query in nearly constant time on average.

This application is quite important, because nearly the same problem appears in [Kruskal's algorithm for finding a minimum spanning tree](#). Using DSU we can [improve](#) the $O(m \log n + n^2)$ complexity to $O(m \log n)$.

Search for connected components in an image

One of the applications of DSU is the following task: there is an image of $n \times m$ pixels. Originally all are white, but then a few black pixels are drawn. You want to determine the size of each white connected component in the final image.

For the solution we simply iterate over all white pixels in the image, for each cell iterate over its four neighbors, and if the neighbor is white call `union_sets`. Thus we will have a DSU with $n \cdot m$ nodes corresponding to image pixels. The resulting trees in the DSU are the desired connected components.

The problem can also be solved by [DFS](#) or [BFS](#), but the method described here has an advantage: it can process the matrix row by row (i.e. to process a row we only need the previous and the current row, and only need a DSU built for the elements of one row) in $O(\min(n, m))$ memory.

Store additional information for each set

DSU allows you to easily store additional information in the sets.

A simple example is the size of the sets: storing the sizes was already described in the Union by size section (the information was stored by the current representative of the set).

In the same way - by storing it at the representative nodes - you can also store any other information about the sets.

Compress jumps along a segment / Painting subarrays offline

One common application of the DSU is the following: There is a set of vertices, and each vertex has an outgoing edge to another vertex. With DSU you can find the end point, to which we get after following all edges from a given starting point, in almost constant time.

A good example of this application is the **problem of painting subarrays**. We have a segment of length L , each element initially has the color 0. We have to repaint the subarray $[l, r]$ with the color c for each query (l, r, c) . At the end we want to find the final color of each cell. We assume that we know all the queries in advance, i.e. the task is offline.

For the solution we can make a DSU, which for each cell stores a link to the next unpainted cell. Thus initially each cell points to itself. After painting one requested repaint of a segment, all cells from that segment will point to the cell after the segment.

Now to solve this problem, we consider the queries **in the reverse order**: from last to first. This way when we execute a query, we only have to paint exactly the unpainted cells in the subarray $[l, r]$. All other cells already contain their final color. To quickly iterate over all unpainted cells, we use the DSU. We find the left-most unpainted cell inside of a segment, repaint it, and with the pointer we move to the next empty cell to the right.

Here we can use the DSU with path compression, but we cannot use union by rank / size (because it is important who becomes the leader after the merge). Therefore the complexity will be $O(\log n)$ per union (which is also quite fast).

Implementation:

```
for (int i = 0; i <= L; i++) {
    make_set(i);
}

for (int i = m-1; i >= 0; i--) {
    int l = query[i].l;
    int r = query[i].r;
    int c = query[i].c;
    for (int v = find_set(l); v <= r; v = find_set(v)) {
        answer[v] = c;
        parent[v] = v + 1;
    }
}
```

There is one optimization: We can use **union by rank**, if we store the next unpainted cell in an additional array `end[]`. Then we can merge two sets into one ranked according to their heuristics, and we obtain the solution in $O(\alpha(n))$.

Support distances up to representative

Sometimes in specific applications of the DSU you need to maintain the distance between a vertex and the representative of its set (i.e. the path length in the tree from the current node to the root of the tree).

If we don't use path compression, the distance is just the number of recursive calls. But this will be inefficient.

However it is possible to do path compression, if we store the **distance to the parent** as additional information for each node.

In the implementation it is convenient to use an array of pairs for `parent[]` and the function `find_set` now returns two numbers: the representative of the set, and the distance to it.

```
void make_set(int v) {
    parent[v] = make_pair(v, 0);
    rank[v] = 0;
}

pair<int, int> find_set(int v) {
    if (v != parent[v].first) {
        int len = parent[v].second;
        parent[v] = find_set(parent[v].first);
        parent[v].second += len;
    }
    return parent[v];
}

void union_sets(int a, int b) {
    a = find_set(a).first;
    b = find_set(b).first;
    if (a != b) {
        if (rank[a] < rank[b])
            swap(a, b);
        parent[b] = make_pair(a, 1);
        if (rank[a] == rank[b])
            rank[a]++;
    }
}
```

Support the parity of the path length / Checking bipartiteness online

In the same way as computing the path length to the leader, it is possible to maintain the parity of the length of the path before him. Why is this application in a separate paragraph?

The unusual requirement of storing the parity of the path comes up in the following task: initially we are given an empty graph, it can be added edges, and we have to answer queries of the form "is the connected component containing this vertex **bipartite**?"

To solve this problem, we make a DSU for storing of the components and store the parity of the path up to the representative for each vertex. Thus we can quickly check if adding an edge leads to a violation of the bipartiteness or not: namely if the ends of the edge lie in the same connected component and have the same parity length to the leader, then adding this edge will produce a cycle of odd length, and the component will lose the bipartiteness property.

The only difficulty that we face is to compute the parity in the `union_find` method.

If we add an edge (a, b) that connects two connected components into one, then when you attach one tree to another we need to adjust the parity.

Let's derive a formula, which computes the parity issued to the leader of the set that will get attached to another set. Let x be the parity of the path length from vertex a up to its leader A , and y as the parity of the path length from vertex b up to its leader B , and t the desired parity that we have to assign to B after the merge. The path consists of the three parts: from B to b , from b to a , which is connected by one edge and therefore has parity 1, and from a to A . Therefore we receive the formula ($\oplus$ denotes the XOR operation):

$$t = x \oplus y \oplus 1$$

Thus regardless of how many joins we perform, the parity of the edges is carried from one leader to another.

We give the implementation of the DSU that supports parity. As in the previous section we use a pair to store the ancestor and the parity. In addition for each set we store in the array `bipartite[]` whether it is still bipartite or not.

```

void make_set(int v) {
    parent[v] = make_pair(v, 0);
    rank[v] = 0;
    bipartite[v] = true;
}

pair<int, int> find_set(int v) {
    if (v != parent[v].first) {
        int parity = parent[v].second;
        parent[v] = find_set(parent[v].first);
        parent[v].second ^= parity;
    }
    return parent[v];
}

void add_edge(int a, int b) {
    pair<int, int> pa = find_set(a);
    a = pa.first;
    int x = pa.second;

    pair<int, int> pb = find_set(b);
    b = pb.first;
    int y = pb.second;

    if (a == b) {
        if (x == y)
            bipartite[a] = false;
    } else {
        if (rank[a] < rank[b])
            swap(a, b);
        parent[b] = make_pair(a, x^y^1);
        bipartite[a] ^= bipartite[b];
        if (rank[a] == rank[b])
            ++rank[a];
    }
}

bool is_bipartite(int v) {
    return bipartite[find_set(v).first];
}

```

Offline RMQ (range minimum query) in $O(\alpha(n))$ on average / Arpa's trick

We are given an array `a[]` and we have to compute some minima in given segments of the array.

The idea to solve this problem with DSU is the following: We will iterate over the array and when we are at the `i`th element we will answer all queries `(L, R)` with `R == i`. To do this efficiently we will keep a DSU using the first `i` elements with the following structure: the parent of an element is the next smaller element to the right of it. Then using this structure the answer to a query will be the `a[find_set(L)]`, the smallest number to the right of `L`.

This approach obviously only works offline, i.e. if we know all queries beforehand.

It is easy to see that we can apply path compression. And we can also use Union by rank, if we store the actual leader in an separate array.

```

struct Query {
    int L, R, idx;
};

vector<int> answer;
vector<vector<Query>> container;

```

`container[i]` contains all queries with `R == i`.

```

stack<int> s;
for (int i = 0; i < n; i++) {
    while (!s.empty() && a[s.top()] > a[i]) {
        parent[s.top()] = i;
        s.pop();
    }
    s.push(i);
    for (Query q : container[i]) {
        answer[q.idx] = a[find_set(q.L)];
    }
}

```

```

    }
}

```

Nowadays this algorithm is known as Arpa's trick. It is named after AmirReza Poorakhavan, who independently discovered and popularized this technique. Although this algorithm existed already before his discovery.

Offline LCA (lowest common ancestor in a tree) in $O(\alpha(n))$ on average

The algorithm for finding the LCA is discussed in the article [Lowest Common Ancestor - Tarjan's off-line algorithm](#). This algorithm compares favorable with other algorithms for finding the LCA due to its simplicity (especially compared to an optimal algorithm like the one from [Farach-Colton and Bender](#)).

Storing the DSU explicitly in a set list / Applications of this idea when merging various data structures

One of the alternative ways of storing the DSU is the preservation of each set in the form of an **explicitly stored list of its elements**. At the same time each element also stores the reference to the representative of his set.

At first glance this looks like an inefficient data structure: by combining two sets we will have to add one list to the end of another and have to update the leadership in all elements of one of the lists.

However it turns out, the use of a **weighting heuristic** (similar to Union by size) can significantly reduce the asymptotic complexity:

$O(m + n \log n)$ to perform m queries on the n elements.

Under weighting heuristic we mean, that we will always **add the smaller of the two sets to the bigger set**. Adding one set to another is easy to implement in `union_sets` and will take time proportional to the size of the added set. And the search for the leader in `find_set` will take $O(1)$ with this method of storing.

Let us prove the **time complexity** $O(m + n \log n)$ for the execution of m queries. We will fix an arbitrary element x and count how often it was touched in the merge operation `union_sets`. When the element x gets touched the first time, the size of the new set will be at least 2. When it gets touched the second time, the resulting set will have size of at least 4, because the smaller set gets added to the bigger one. And so on. This means, that x can only be moved in at most $\log n$ merge operations. Thus the sum over all vertices gives $O(n \log n)$ plus $O(1)$ for each request.

Here is an implementation:

```

vector<int> lst[MAXN];
int parent[MAXN];

void make_set(int v) {
    lst[v] = vector<int>(1, v);
    parent[v] = v;
}

int find_set(int v) {
    return parent[v];
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (lst[a].size() < lst[b].size())
            swap(a, b);
        while (!lst[b].empty()) {
            int v = lst[b].back();
            lst[b].pop_back();
            parent[v] = a;
            lst[a].push_back(v);
        }
    }
}

```

This idea of adding the smaller part to a bigger part can also be used in a lot of solutions that have nothing to do with DSU.

For example consider the following **problem**: we are given a tree, each leaf has a number assigned (same number can appear multiple times on different leaves). We want to compute the number of different numbers in the subtree for every node of the tree.

Applying to this task the same idea it is possible to obtain this solution: we can implement a [DFS](#), which will return a pointer to a set of integers - the list of numbers in that subtree. Then to get the answer for the current node (unless of course it is a leaf), we call DFS for all children of that node, and merge all the received sets together. The size of the resulting set will be the answer for the current node. To efficiently combine multiple sets we just apply the above-described recipe: we merge the sets by simply adding smaller ones to larger. In the end we get a $O(n \log^2 n)$ solution, because one number will only be added to a set at most $O(\log n)$ times.

Storing the DSU by maintaining a clear tree structure / Online bridge finding in $O(\alpha(n))$ on average

One of the most powerful applications of DSU is that it allows you to store both as **compressed and uncompressed trees**. The compressed form can be used for merging of trees and for the verification if two vertices are in the same tree, and the uncompressed form can be used - for example - to search for paths between two given vertices, or other traversals of the tree structure.

In the implementation this means that in addition to the compressed ancestor array `parent[]` we will need to keep the array of uncompressed ancestors `real_parent[]`. It is trivial that maintaining this additional array will not worsen the complexity: changes in it only occur when we merge two trees, and only in one element.

On the other hand when applied in practice, we often need to connect trees using a specified edge other than that using the two root nodes. This means that we have no other choice but to re-root one of the trees (make the ends of the edge the new root of the tree).

At first glance it seems that this re-rooting is very costly and will greatly worsen the time complexity. Indeed, for rooting a tree at vertex v we must go from the vertex to the old root and change directions in `parent[]` and `real_parent[]` for all nodes on that path.

However in reality it isn't so bad, we can just re-root the smaller of the two trees similar to the ideas in the previous sections, and get $O(\log n)$ on average.

More details (including proof of the time complexity) can be found in the article [Finding Bridges Online](#).

Historical retrospective

The data structure DSU has been known for a long time.

This way of storing this structure in the form of a **forest of trees** was apparently first described by Galler and Fisher in 1964 (Galler, Fisher, "An Improved Equivalence Algorithm"), however the complete analysis of the time complexity was conducted much later.

The optimizations path compression and Union by rank have been developed by McIlroy and Morris, and independently of them also by Titter.

Hopcroft and Ullman showed in 1973 the time complexity $O(\log^* n)$ (Hopcroft, Ullman "Set-merging algorithms") - here $\log^*$ is the **iterated logarithm** (this is a slow-growing function, but still not as slow as the inverse Ackermann function).

For the first time the evaluation of $O(\alpha(n))$ was shown in 1975 (Tarjan "Efficiency of a Good But Not Linear Set Union Algorithm"). Later in 1985 he, along with Leeuwen, published multiple complexity analyses for several different rank heuristics and ways of compressing the path (Tarjan, Leeuwen "Worst-case Analysis of Set Union Algorithms").

Finally in 1989 Fredman and Sachs proved that in the adopted model of computation **any** algorithm for the disjoint set union problem has to work in at least $O(\alpha(n))$ time on average (Fredman, Saks, "The cell probe complexity of dynamic data structures").

Problems

- [TIMUS - Anansi's Cobweb](#)
- [Codeforces - Roads not only in Berland](#)
- [TIMUS - Parity](#)
- [SPOJ - Strange Food Chain](#)
- [SPOJ - COLORFUL ARRAY](#)
- [SPOJ - Consecutive Letters](#)
- [Toph - Unbelievable Array](#)
- [HackerEarth - Lexicographically minimal string](#)
- [HackerEarth - Fight in Ninja World](#)

Contributors:

[Jakob Kogler](#) (89.36%) [Alexander Kernozhitsky](#) (3.55%) [wilwxk](#) (2.87%) [Oleksandr Kulkov](#) (1.18%) [Tahanima Chowdhury](#) (0.84%) [Ankit Sangwan](#) (0.34%)
[Tanmay Sinha](#) (0.34%) [gampu](#) (0.34%) [Abdallah Ahmed](#) (0.17%) [Ankush Khanna](#) (0.17%) [Cameron Custer](#) (0.17%) [Shubham Phapale](#) (0.17%)
[Vedant Karandikar](#) (0.17%) [platypusperry23](#) (0.17%) [wiktorkuchta](#) (0.17%)

Translated

From: e-maxx.ru

Fenwick Tree

Let f be some group operation (a binary associative function over a set with an identity element and inverse elements) and A be an array of integers of length N . Denote f 's infix notation as $*$ that is, $f(x, y) = x * y$ for arbitrary integers x, y . (Since this is associative, we will omit parentheses for order of application of f when using infix notation.)

The Fenwick tree is a data structure which:

- calculates the value of function f in the given range $[l, r]$ (i.e. $A_l * A_{l+1} * \dots * A_r$) in $O(\log N)$ time
- updates the value of an element of A in $O(\log N)$ time
- requires $O(N)$ memory (the same amount required for A)
- is easy to use and code, especially in the case of multidimensional arrays

The most common application of a Fenwick tree is *calculating the sum of a range*. For example, using addition over the set of integers as the group operation, i.e. $f(x, y) = x + y$: the binary operation, $*$ is $+$ in this case, so $A_l * A_{l+1} * \dots * A_r = A_l + A_{l+1} + \dots + A_r$.

The Fenwick tree is also called a **Binary Indexed Tree (BIT)**. It was first described in a paper titled "A new data structure for cumulative frequency tables" (Peter M. Fenwick, 1994).

Description

Overview

For the sake of simplicity, we will assume that function f is defined as $f(x, y) = x + y$ over the integers.

Suppose we are given an array of integers, $A[0 \dots N - 1]$. (Note that we are using zero-based indexing.) A Fenwick tree is just an array, $T[0 \dots N - 1]$, where each element is equal to the sum of elements of A in some range, $[g(i), i]$:

$$T_i = \sum_{j=g(i)}^i A_j$$

where g is some function that satisfies $0 \leq g(i) \leq i$. We will define g in the next few paragraphs.

The data structure is called a tree because there is a nice representation of it in the form of a tree, although we don't need to model an actual tree with nodes and edges. We only need to maintain the array T to handle all queries.

Note: The Fenwick tree presented here uses zero-based indexing. Many people use a version of the Fenwick tree that uses one-based indexing. As such, you will also find an alternative implementation which uses one-based indexing in the implementation section. Both versions are equivalent in terms of time and memory complexity.

Now we can write some pseudo-code for the two operations mentioned above. Below, we get the sum of elements of A in the range $[0, r]$ and update (increase) some element A_i :

```
def sum(int r):
    res = 0
    while (r >= 0):
        res += t[r]
        r = g(r) - 1
    return res

def increase(int i, int delta):
    for all j with g(j) <= i <= j:
        t[j] += delta
```

The function `sum` works as follows:

1. First, it adds the sum of the range $[g(r), r]$ (i.e. $T[r]$) to the `result`.
2. Then, it "jumps" to the range $[g(g(r) - 1), g(r) - 1]$ and adds this range's sum to the `result`.
3. This continues until it "jumps" from $[0, g(g(\dots g(r) - 1 \dots - 1) - 1)]$ to $[g(-1), -1]$; this is where the `sum` function stops jumping.

The function `increase` works with the same analogy, but it "jumps" in the direction of increasing indices:

1. The sum for each range of the form $[g(j), j]$ which satisfies the condition $g(j) \leq i \leq j$ is increased by `delta`; that is, `t[j] += delta`. Therefore, it updates all elements in T that correspond to ranges in which A_i lies.

The complexity of both `sum` and `increase` depend on the function g . There are many ways to choose the function g such that $0 \leq g(i) \leq i$ for all i . For instance, the function $g(i) = i$ works, which yields $T = A$ (in which case, the summation queries are slow). We could also take the function $g(i) = 0$. This would correspond to prefix sum arrays (in which case, finding the sum of the range $[0, i]$ will only take constant time; however, updates are slow). The clever part of the algorithm for Fenwick trees is how it uses a special definition of the function g which can handle both operations in $O(\log N)$ time.

Definition of $g(i)$

The computation of $g(i)$ is defined using the following simple operation: we replace all trailing 1 bits in the binary representation of i with 0 bits.

In other words, if the least significant digit of i in binary is 0, then $g(i) = i$. And otherwise the least significant digit is a 1, and we take this 1 and all other trailing 1s and flip them.

For instance we get

$$g(11) = g(1011_2) = 1000_2 = 8$$

$$g(12) = g(1100_2) = 1100_2 = 12$$

$$g(13) = g(1101_2) = 1100_2 = 12$$

$$g(14) = g(1110_2) = 1110_2 = 14$$

$$g(15) = g(1111_2) = 0000_2 = 0$$

There exists a simple implementation using bitwise operations for the non-trivial operation described above:

$$g(i) = i \& (i + 1),$$

where $\&$ is the bitwise AND operator. It is not hard to convince yourself that this solution does the same thing as the operation described above.

Now, we just need to find a way to iterate over all j 's, such that $g(j) \leq i \leq j$.

It is easy to see that we can find all such j 's by starting with i and flipping the last unset bit. We will call this operation $h(j)$. For example, for $i = 10$ we have:

$$10 = 0001010_2$$

$$h(10) = 11 = 0001011_2$$

$$h(11) = 15 = 0001111_2$$

$$h(15) = 31 = 0011111_2$$

$$h(31) = 63 = 0111111_2$$

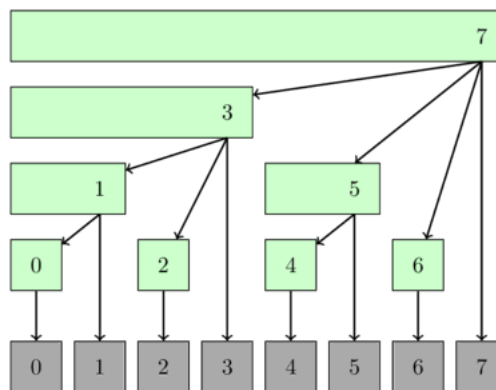
⋮

Unsurprisingly, there also exists a simple way to perform h using bitwise operations:

$$h(j) = j \mid (j + 1),$$

where $\mid$ is the bitwise OR operator.

The following image shows a possible interpretation of the Fenwick tree as tree. The nodes of the tree show the ranges they cover.



Implementation

Finding sum in one-dimensional array

Here we present an implementation of the Fenwick tree for sum queries and single updates.

The normal Fenwick tree can only answer sum queries of the type $[0, r]$ using `sum(int r)`, however we can also answer other queries of the type $[l, r]$ by computing two sums $[0, r]$ and $[0, l - 1]$ and subtract them. This is handled in the `sum(int l, int r)` method.

Also this implementation supports two constructors. You can create a Fenwick tree initialized with zeros, or you can convert an existing array into the Fenwick form.

```
struct FenwickTree {
    vector<int> bit; // binary indexed tree
    int n;

    FenwickTree(int n) {
        this->n = n;
        bit.assign(n, 0);
    }

    FenwickTree(vector<int> const &a) : FenwickTree(a.size()) {
        for (size_t i = 0; i < a.size(); i++)
            add(i, a[i]);
    }
};
```

```

    }

    int sum(int r) {
        int ret = 0;
        for (; r >= 0; r = (r & (r + 1)) - 1)
            ret += bit[r];
        return ret;
    }

    int sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }

    void add(int idx, int delta) {
        for (; idx < n; idx = idx | (idx + 1))
            bit[idx] += delta;
    }
};

```

Linear construction

The above implementation requires $O(N \log N)$ time. It's possible to improve that to $O(N)$ time.

The idea is, that the number $a[i]$ at index i will contribute to the range stored in $bit[i]$, and to all ranges that the index $i|(i + 1)$ contributes to. So by adding the numbers in order, you only have to push the current sum further to the next range, where it will then get pushed further to the next range, and so on.

```

FenwickTree(vector<int> const &a) : FenwickTree(a.size()){
    for (int i = 0; i < n; i++) {
        bit[i] += a[i];
        int r = i | (i + 1);
        if (r < n) bit[r] += bit[i];
    }
}

```

Finding minimum of $[0, r]$ in one-dimensional array

It is obvious that there is no easy way of finding minimum of range $[l, r]$ using Fenwick tree, as Fenwick tree can only answer queries of type $[0, r]$. Additionally, each time a value is update'd, the new value has to be smaller than the current value. Both significant limitations are because the `min` operation together with the set of integers doesn't form a group, as there are no inverse elements.

```

struct FenwickTreeMin {
    vector<int> bit;
    int n;
    const int INF = (int)1e9;

    FenwickTreeMin(int n) {
        this->n = n;
        bit.assign(n, INF);
    }

    FenwickTreeMin(vector<int> a) : FenwickTreeMin(a.size()) {
        for (size_t i = 0; i < a.size(); i++)
            update(i, a[i]);
    }

    int getmin(int r) {
        int ret = INF;
        for (; r >= 0; r = (r & (r + 1)) - 1)
            ret = min(ret, bit[r]);
        return ret;
    }

    void update(int idx, int val) {
        for (; idx < n; idx = idx | (idx + 1))
            bit[idx] = min(bit[idx], val);
    }
};

```

Note: it is possible to implement a Fenwick tree that can handle arbitrary minimum range queries and arbitrary updates. The paper [Efficient Range Minimum Queries using Binary Indexed Trees](#) describes such an approach. However with that approach you need to maintain a second

binary indexed tree over the data, with a slightly different structure, since one tree is not enough to store the values of all elements in the array. The implementation is also a lot harder compared to the normal implementation for sums.

Finding sum in two-dimensional array

As claimed before, it is very easy to implement Fenwick Tree for multidimensional array.

```
struct FenwickTree2D {
    vector<vector<int>> bit;
    int n, m;

    // init(...) { ... }

    int sum(int x, int y) {
        int ret = 0;
        for (int i = x; i >= 0; i = (i & (i + 1)) - 1)
            for (int j = y; j >= 0; j = (j & (j + 1)) - 1)
                ret += bit[i][j];
        return ret;
    }

    void add(int x, int y, int delta) {
        for (int i = x; i < n; i = i | (i + 1))
            for (int j = y; j < m; j = j | (j + 1))
                bit[i][j] += delta;
    }
};
```

One-based indexing approach

For this approach we change the requirements and definition for $T[]$ and $g()$ a little bit. We want $T[i]$ to store the sum of $[g(i) + 1; i]$. This changes the implementation a little bit, and allows for a similar nice definition for $g(i)$:

```
def sum(int r):
    res = 0
    while (r > 0):
        res += t[r]
        r = g(r)
    return res

def increase(int i, int delta):
    for all j with g(j) < i <= j:
        t[j] += delta
```

The computation of $g(i)$ is defined as: toggling of the last set 1 bit in the binary representation of i .

$$g(7) = g(111_2) = 110_2 = 6$$

$$g(6) = g(110_2) = 100_2 = 4$$

$$g(4) = g(100_2) = 000_2 = 0$$

The last set bit can be extracted using $i \& (-i)$, so the operation can be expressed as:

$$g(i) = i - (i \& (-i)).$$

And it's not hard to see, that you need to change all values $T[j]$ in the sequence $i, h(i), h(h(i)), \dots$ when you want to update $A[j]$, where $h(i)$ is defined as:

$$h(i) = i + (i \& (-i)).$$

As you can see, the main benefit of this approach is that the binary operations complement each other very nicely.

The following implementation can be used like the other implementations, however it uses one-based indexing internally.

```
struct FenwickTreeOneBasedIndexing {
    vector<int> bit; // binary indexed tree
    int n;

    FenwickTreeOneBasedIndexing(int n) {
        this->n = n + 1;
        bit.assign(n + 1, 0);
    }

    FenwickTreeOneBasedIndexing(vector<int> a)
        : FenwickTreeOneBasedIndexing(a.size()) {
        for (size_t i = 0; i < a.size(); i++)
            add(i, a[i]);
    }

    int sum(int idx) {
        int ret = 0;
        for (++idx; idx > 0; idx -= idx & -idx)
            ret += bit[idx];
        return ret;
    }

    int sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }

    void add(int idx, int delta) {
        for (++idx; idx < n; idx += idx & -idx)
            bit[idx] += delta;
    }
};
```

Range operations

A Fenwick tree can support the following range operations:

1. Point Update and Range Query
2. Range Update and Point Query
3. Range Update and Range Query

1. Point Update and Range Query

This is just the ordinary Fenwick tree as explained above.

2. Range Update and Point Query

Using simple tricks we can also do the reverse operations: increasing ranges and querying for single values.

Let the Fenwick tree be initialized with zeros. Suppose that we want to increment the interval $[l, r]$ by x . We make two point update operations on Fenwick tree which are `add(l, x)` and `add(r+1, -x)`.

If we want to get the value of $A[i]$, we just need to take the prefix sum using the ordinary range sum method. To see why this is true, we can just focus on the previous increment operation again. If $i < l$, then the two update operations have no effect on the query and we get the sum 0. If $i \in [l, r]$, then we get the answer x because of the first update operation. And if $i > r$, then the second update operation will cancel the effect of first one.

The following implementation uses one-based indexing.

```
void add(int idx, int val) {
    for (++idx; idx < n; idx += idx & -idx)
        bit[idx] += val;
}

void range_add(int l, int r, int val) {
    add(l, val);
    add(r + 1, -val);
}
```

```

int point_query(int idx) {
    int ret = 0;
    for (++idx; idx > 0; idx -= idx & -idx)
        ret += bit[idx];
    return ret;
}

```

Note: of course it is also possible to increase a single point $A[i]$ with `range_add(i, i, val)`.

3. Range Update and Range Query

To support both range updates and range queries we will use two BITs namely $B_1[]$ and $B_2[]$, initialized with zeros.

Suppose that we want to increment the interval $[l, r]$ by the value x . Similarly as in the previous method, we perform two point updates on B_1 : `add(B1, l, x)` and `add(B1, r+1, -x)`. And we also update B_2 . The details will be explained later.

```

def range_add(l, r, x):
    add(B1, l, x)
    add(B1, r+1, -x)
    add(B2, l, x*(l-1))
    add(B2, r+1, -x*r)

```

After the range update (l, r, x) the range sum query should return the following values:

$$\begin{aligned}
 & r \cdot 0 & i < l \\
 \text{sum}[0, i] = & \begin{cases} x \cdot (i - (l - 1)) & l \leq i \leq r \\ x \cdot (r - l + 1) & i > r \end{cases}
 \end{aligned}$$

We can write the range sum as difference of two terms, where we use B_1 for first term and B_2 for second term. The difference of the queries will give us prefix sum over $[0, i]$.

$$\begin{aligned}
 \text{sum}[0, i] &= \text{sum}(B_1, i) \cdot i - \text{sum}(B_2, i) \\
 & r \cdot 0 \cdot i - 0 & i < l \\
 & = \begin{cases} x \cdot i - x \cdot (l - 1) & l \leq i \leq r \\ 0 \cdot i - (x \cdot (l - 1) - x \cdot r) & i > r \end{cases}
 \end{aligned}$$

The last expression is exactly equal to the required terms. Thus we can use B_2 for shaving off extra terms when we multiply $B_1[i] \times i$.

We can find arbitrary range sums by computing the prefix sums for $l - 1$ and r and taking the difference of them again.

```

def add(b, idx, x):
    while idx <= N:
        b[idx] += x
        idx += idx & -idx

def range_add(l, r, x):
    add(B1, l, x)
    add(B1, r+1, -x)
    add(B2, l, x*(l-1))
    add(B2, r+1, -x*r)

def sum(b, idx):
    total = 0
    while idx > 0:
        total += b[idx]
        idx -= idx & -idx
    return total

```

```
def prefix_sum(idx):  
    return sum(B1, idx)*idx - sum(B2, idx)  
  
def range_sum(l, r):  
    return prefix_sum(r) - prefix_sum(l-1)
```

Practice Problems

- [UVA 12086 - Potentiometers](#)
- [LOJ 1112 - Curious Robin Hood](#)
- [LOJ 1266 - Points in Rectangle](#)
- [Codechef - SPREAD](#)
- [SPOJ - CTRICK](#)
- [SPOJ - MATSUM](#)
- [SPOJ - DQUERY](#)
- [SPOJ - NKTEAM](#)
- [SPOJ - YODANESS](#)
- [SRM 310 - FloatingMedian](#)
- [SPOJ - Ada and Behives](#)
- [Hackerearth - Counting in Byteland](#)
- [DevSkill - Shan and String \(archived\)](#)
- [Codeforces - Little Artem and Time Machine](#)
- [Codeforces - Hanoi Factory](#)
- [SPOJ - Tulip and Numbers](#)
- [SPOJ - SUMSUM](#)
- [SPOJ - Sabir and Gifts](#)
- [SPOJ - The Permutation Game Again](#)
- [SPOJ - Zig when you Zag](#)
- [SPOJ - Cryon](#)
- [SPOJ - Weird Points](#)
- [SPOJ - Its a Murder](#)
- [SPOJ - Bored of Suffixes and Prefixes](#)
- [SPOJ - Mega Inversions](#)
- [Codeforces - Subsequences](#)
- [Codeforces - Ball](#)
- [GYM - The Kamphaeng Phet's Chedis](#)
- [Codeforces - Garlands](#)
- [Codeforces - Inversions after Shuffle](#)
- [GYM - Cairo Market](#)
- [Codeforces - Goodbye Souvenir](#)
- [SPOJ - Ada and Species](#)
- [Codeforces - Thor](#)
- [CSES - Forest Queries II](#)
- [Latin American Regionals 2017 - Fundraising](#)

Other sources

- [Fenwick tree on Wikipedia](#)
- [Binary indexed trees tutorial on TopCoder](#)
- [Range updates and queries](#)

Contributors:

[Jakob Kogler](#) (36.98%) [Madhur Chauhan](#) (22.66%) [Trung, Nguyen Thanh](#) (20.87%) [JJCUBER](#) (7.36%) [Morass](#) (4.37%) [Tanmay Sinha](#) (1.59%)
[Oleksandr Kulkov](#) (1.19%) [Vatsal Sharma](#) (1.19%) [Araf Al-Jami](#) (0.99%) [Mariia Mykhailova](#) (0.6%) [Michael Hayter](#) (0.6%) [Joaquin Garmendia Cabrera](#) (0.4%)
[Zyad M. Ayad](#) (0.4%) [Anurag Srivastava](#) (0.2%) [Harshit Jain](#) (0.2%) [Michael Zegar](#) (0.2%) [Turf Auliarachman](#) (0.2%)

Translated From: e-maxx.ru

Sqrt Decomposition

Sqrt Decomposition is a method (or a data structure) that allows you to perform some common operations (finding sum of the elements of the sub-array, finding the minimal/maximal element, etc.) in $O(\sqrt{n})$ operations, which is much faster than $O(n)$ for the trivial algorithm.

First we describe the data structure for one of the simplest applications of this idea, then show how to generalize it to solve some other problems, and finally look at a slightly different use of this idea: splitting the input requests into sqrt blocks.

Sqrt-decomposition based data structure

Given an array $a[0 \dots n-1]$, implement a data structure that allows to find the sum of the elements $a[l \dots r]$ for arbitrary l and r in $O(\sqrt{n})$ operations.

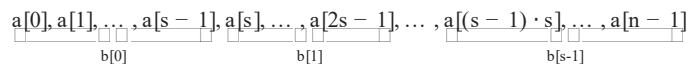
Description

The basic idea of sqrt decomposition is preprocessing. We'll divide the array a into blocks of length approximately $\sqrt{n}$, and for each block i we'll precalculate the sum of elements in it $b[i]$.

We can assume that both the size of the block and the number of blocks are equal to $\sqrt{n}$ rounded up:

$$s = \lceil \sqrt{n} \rceil$$

Then the array a is divided into blocks in the following way:



The last block may have fewer elements than the others (if n not a multiple of s), it is not important to the discussion (as it can be handled easily). Thus, for each block k , we know the sum of elements on it $b[k]$:

$$b[k] = \sum_{i=k*s}^{\min(n-1, (k+1)*s-1)} a[i]$$

So, we have calculated the values of $b[k]$ (this required $O(n)$ operations). How can they help us to answer each query $[l, r]$? Notice that if the interval $[l, r]$ is long enough, it will contain several whole blocks, and for those blocks we can find the sum of elements in them in a single operation. As a result, the interval $[l, r]$ will contain parts of only two blocks, and we'll have to calculate the sum of elements in these parts trivially.

Thus, in order to calculate the sum of elements on the interval $[l, r]$ we only need to sum the elements of the two "tails": $[l \dots (k+1)*s-1]$ and $[p*s \dots r]$, and sum the values $b[i]$ in all the blocks from $k+1$ to $p-1$:

$$\sum_{i=l}^r a[i] = \sum_{i=l}^{(k+1)*s-1} a[i] + \sum_{i=k+1}^{p-1} b[i] + \sum_{i=p*s}^r a[i]$$

Note: When $k = p$, i.e. l and r belong to the same block, the formula can't be applied, and the sum should be calculated trivially.

This approach allows us to significantly reduce the number of operations. Indeed, the size of each "tail" does not exceed the block length s , and the number of blocks in the sum does not exceed s . Since we have chosen $s \approx \sqrt{n}$, the total number of operations required to find the sum of elements on the interval $[l, r]$ is $O(\sqrt{n})$.

Implementation

Let's start with the simplest implementation:

```
// input data
int n;
vector<int> a (n);

// preprocessing
int len = (int) sqrt (n + .0) + 1; // size of the block and the number of blocks
vector<int> b (len);
for (int i=0; i<n; ++i)
    b[i / len] += a[i];

// answering the queries
for (;;) {
    int l, r;
    // read input data for the next query
    int sum = 0;
    for (int i=l; i<=r; )
        if (i % len == 0 && i + len - 1 <= r) {
            // if the whole block starting at i belongs to [l, r]
            sum += b[i / len];
            i += len;
        }
        else {
            sum += a[i];
            ++i;
        }
}
```

This implementation has unreasonably many division operations (which are much slower than other arithmetical operations). Instead, we can calculate the indices of the blocks c_l and c_r which contain indices l and r , and loop through blocks $c_l + 1 \dots c_r - 1$ with separate processing of the "tails" in blocks c_l and c_r . This approach corresponds to the last formula in the description, and makes the case $c_l = c_r$ a special case.

```
int sum = 0;
int c_l = l / len,    c_r = r / len;
if (c_l == c_r)
    for (int i=l; i<=r; ++i)
        sum += a[i];
else {
    for (int i=l, end=(c_l+1)*len-1; i<=end; ++i)
        sum += a[i];
    for (int i=c_l+1; i<=c_r-1; ++i)
        sum += b[i];
    for (int i=c_r*len; i<=r; ++i)
        sum += a[i];
}
```

Other problems

So far we were discussing the problem of finding the sum of elements of a continuous subarray. This problem can be extended to allow to **update individual array elements**. If an element $a[i]$ changes, it's sufficient to update the value of $b[k]$ for the block to which this element belongs ($k = i/s$) in one operation:

$$b[k]_{+} = a_{\text{new}}[i] - a_{\text{old}}[i]$$

On the other hand, the task of finding the sum of elements can be replaced with the task of finding minimal/maximal element of a subarray. If this problem has to address individual elements' updates as well, updating the value of $b[k]$ is also possible, but it will require iterating through all values of block k in $O(s) = O(\sqrt{n})$ operations.

Sqrt decomposition can be applied in a similar way to a whole class of other problems: finding the number of zero elements, finding the first non-zero element, counting elements which satisfy a certain property etc.

Another class of problems appears when we need to **update array elements on intervals**: increment existing elements or replace them with a given value.

For example, let's say we can do two types of operations on an array: add a given value δ to all array elements on interval $[l, r]$ or query the value of element $a[i]$. Let's store the value which has to be added to all elements of block k in $b[k]$ (initially all $b[k] = 0$). During each "add" operation we need to add δ to $b[k]$ for all blocks which belong to interval $[l, r]$ and to add δ to $a[i]$ for all elements which belong to the "tails" of the interval. The answer to query i is simply $a[i] + b[i/s]$. This way "add" operation has $O(\sqrt{n})$ complexity, and answering a query has $O(1)$ complexity.

Finally, those two classes of problems can be combined if the task requires doing **both** element updates on an interval and queries on an interval. Both operations can be done with $O(\sqrt{n})$ complexity. This will require two block arrays b and c : one to keep track of element updates and another to keep track of answers to the query. An example of how to do this is shown below.

```
// NOTE: MANUALLY ADDED BY ME
class SqrtDecomp {
    vector<int> a, b, c;
    int len, block_count;

public:
    SqrtDecomp(vector<int>& input) {
        len = sqrt(input.size());
        this->a = input;
        this->a.resize(input.size() + len, 0);
        block_count = (input.size() / len) + 1;
        b.resize(block_count, 0);
        c.resize(block_count, 0); // an additional c[block] is added when querying elements in block
        for (int i = 0; i < block_count; ++i) { // compute block sums
            b[i] = accumulate(a.begin() + i * len, a.begin() + (i + 1) * len, 0);
        }
    }

    void update(int l, int r, int diff) {
        int c_l = (l / len) + 1;
        int c_r = (r / len) - 1;
        if (c_l > c_r) { // doesn't cover any blocks
            for (int i = l; i <= r; ++i) {
                a[i] += diff;
                b[i / len] += diff;
            }
            return;
        }

        for (int i = c_l; i <= c_r; ++i) { // update blocks
            c[i] += diff;
            b[i] += diff * len;
        }

        for (int i = l; i < c_l * len; ++i) { // update individual cells
            a[i] += diff;
            b[c_l - 1] += diff;
        }

        for (int i = (c_r + 1) * len; i <= r; ++i) {
            a[i] += diff;
            b[c_r + 1] += diff;
        }
    }

    int query(int l, int r) {
        int c_l = (l / len) + 1;
        int c_r = (r / len) - 1;
        int sum = 0;

        if (c_l > c_r) { // doesn't cover any blocks
            for (int i = l; i <= r; ++i) {
                sum += a[i] + c[i / len];
            }
            return sum;
        }

        for (int i = c_l; i <= c_r; ++i) { // add value from blocks
            sum += b[i];
        }
    }
};
```

```

    }
    for (int i = l; i < c_l * len; ++i) { // add value from individual cells
        sum += a[i] + c[c_l - 1];
    }
    for (int i = (c_r + 1) * len; i <= r; ++i) {
        sum += a[i] + c[c_r + 1];
    }
    return sum;
}
};

```

We can now perform updates and queries on an interval.

```

vector<int> a(N);
// create a from input

SqrtDecomp sq(a);
int l, r, x;
// read input data for the next query
sq.update(l, r, x); // add x to every element on the interval [l, r]
cout << sq.query(l, r) << "\n"; // query the sum of elements in the interval [l, r]

```

There exist other problems which can be solved using sqrt decomposition, for example, a problem about maintaining a set of numbers which would allow adding/deleting numbers, checking whether a number belongs to the set and finding k-th largest number. To solve it one has to store numbers in increasing order, split into several blocks with $\sqrt{n}$ numbers in each. Every time a number is added/deleted, the blocks have to be rebalanced by moving numbers between beginnings and ends of adjacent blocks.

Mo's algorithm

A similar idea, based on sqrt decomposition, can be used to answer range queries (Q) offline in $O((N + Q)\sqrt{N})$. This might sound like a lot worse than the methods in the previous section, since this is a slightly worse complexity than we had earlier and cannot update values between two queries. But in a lot of situations this method has advantages. During a normal sqrt decomposition, we have to precompute the answers for each block, and merge them during answering queries. In some problems this merging step can be quite problematic. E.g. when each queries asks to find the **mode** of its range (the number that appears the most often). For this each block would have to store the count of each number in it in some sort of data structure, and we can no longer perform the merge step fast enough any more. **Mo's algorithm** uses a completely different approach, that can answer these kind of queries fast, because it only keeps track of one data structure, and the only operations with it are easy and fast.

The idea is to answer the queries in a special order based on the indices. We will first answer all queries which have the left index in block 0, then answer all queries which have left index in block 1 and so on. And also we will have to answer the queries of a block in a special order, namely sorted by the right index of the queries.

As already said we will use a single data structure. This data structure will store information about the range. At the beginning this range will be empty. When we want to answer the next query (in the special order), we simply extend or reduce the range, by adding/removing elements on both sides of the current range, until we transformed it into the query range. This way, we only need to add or remove a single element once at a time, which should be pretty easy operations in our data structure.

Since we change the order of answering the queries, this is only possible when we are allowed to answer the queries in offline mode.

Implementation

In Mo's algorithm we use two functions for adding an index and for removing an index from the range which we are currently maintaining.

```

void remove(idx); // TODO: remove value at idx from data structure
void add(idx);    // TODO: add value at idx from data structure
int get_answer(); // TODO: extract the current answer of the data structure

int block_size;

struct Query {
    int l, r, idx;
    bool operator<(Query other) const
    {
        return make_pair(l / block_size, r) <
               make_pair(other.l / block_size, other.r);
    }
}

```

```

    }
};

vector<int> mo_s_algorithm(vector<Query> queries) {
    vector<int> answers(queries.size());
    sort(queries.begin(), queries.end());

    // TODO: initialize data structure

    int cur_l = 0;
    int cur_r = -1;
    // invariant: data structure will always reflect the range [cur_l, cur_r]
    for (Query q : queries) {
        while (cur_l > q.l) {
            cur_l--;
            add(cur_l);
        }
        while (cur_r < q.r) {
            cur_r++;
            add(cur_r);
        }
        while (cur_l < q.l) {
            remove(cur_l);
            cur_l++;
        }
        while (cur_r > q.r) {
            remove(cur_r);
            cur_r--;
        }
        answers[q.idx] = get_answer();
    }
    return answers;
}

```

Based on the problem we can use a different data structure and modify the `add` / `remove` / `get_answer` functions accordingly. For example if we are asked to find range sum queries then we use a simple integer as data structure, which is 0 at the beginning. The `add` function will simply add the value of the position and subsequently update the answer variable. On the other hand `remove` function will subtract the value at position and subsequently update the answer variable. And `get_answer` just returns the integer.

For answering mode-queries, we can use a binary search tree (e.g. `map<int, int>`) for storing how often each number appears in the current range, and a second binary search tree (e.g. `set<pair<int, int>>`) for keeping counts of the numbers (e.g. as count-number pairs) in order. The `add` method removes the current number from the second BST, increases the count in the first one, and inserts the number back into the second one. `remove` does the same thing, it only decreases the count. And `get_answer` just looks at second tree and returns the best value in $O(1)$.

Complexity

Sorting all queries will take $O(Q \log Q)$.

How about the other operations? How many times will the `add` and `remove` be called?

Let's say the block size is S .

If we only look at all queries having the left index in the same block, the queries are sorted by the right index. Therefore we will call `add(cur_r)` and `remove(cur_r)` only $O(N)$ times for all these queries combined. This gives $O(\frac{N}{S}N)$ calls for all blocks.

The value of `cur_l` can change by at most $O(S)$ during between two queries. Therefore we have an additional $O(SQ)$ calls of `add(cur_l)` and `remove(cur_l)`.

For $S \approx \sqrt{N}$ this gives $O((N + Q)\sqrt{N})$ operations in total. Thus the complexity is $O((N + Q)F\sqrt{N})$ where $O(F)$ is the complexity of `add` and `remove` function.

Tips for improving runtime

- Block size of precisely $\sqrt{N}$ doesn't always offer the best runtime. For example, if $\sqrt{N} = 750$ then it may happen that block size of 700 or 800 may run better. More importantly, don't compute the block size at runtime - make it `const`. Division by constants is well optimized by

compilers.

- In odd blocks sort the right index in ascending order and in even blocks sort it in descending order. This will minimize the movement of right pointer, as the normal sorting will move the right pointer from the end back to the beginning at the start of every block. With the improved version this resetting is no more necessary.

```
bool cmp(pair<int, int> p, pair<int, int> q) {
    if (p.first / BLOCK_SIZE != q.first / BLOCK_SIZE)
        return p < q;
    return (p.first / BLOCK_SIZE & 1) ? (p.second < q.second) : (p.second > q.second);
}
```

You can read about even faster sorting approach [here](#).

Practice Problems

- [Codeforces - Kuriyama Mirai's Stones](#)
- [UVA - 12003 - Array Transformer](#)
- [UVA - 11990 Dynamic Inversion](#)
- [SPOJ - Give Away](#)
- [Codeforces - Till I Collapse](#)
- [Codeforces - Destiny](#)
- [Codeforces - Holes](#)
- [Codeforces - XOR and Favorite Number](#)
- [Codeforces - Powerful array](#)
- [SPOJ - DQUERY](#)
- [Codeforces - Robin Hood Archery](#)

Contributors:

[Madhur Chauhan](#) (35.15%)
 [Mariia Mykhailova](#) (26.06%)
 [Your name](#) (24.85%)
 [Rodion Gorkovenko](#) (3.33%)
 [Morass](#) (2.73%)
 [Oleksandr Kulkov](#) (2.42%)
[Tanmay Sinha](#) (1.52%)
[gampu](#) (0.91%)
[wiktorkuchta](#) (0.61%)
[Abir Chakraborty](#) (0.3%)
[Anton Algmyr](#) (0.3%)
[Avinash Pandey](#) (0.3%)
[Jakob Kogler](#) (0.3%)
[Michael Zegar](#) (0.3%)
[Pratik Yadav](#) (0.3%)
[Rajat Biswas](#) (0.3%)
[Soumik Sarkar](#) (0.3%)

Translated From: e-maxx.ru

Segment Tree

A Segment Tree is a data structure that stores information about array intervals as a tree. This allows answering range queries over an array efficiently, while still being flexible enough to allow quick modification of the array. This includes finding the sum of consecutive array elements $a[l \dots r]$, or finding the minimum element in a such a range in $O(\log n)$ time. Between answering such queries, the Segment Tree allows modifying the array by replacing one element, or even changing the elements of a whole subsegment (e.g. assigning all elements $a[l \dots r]$ to any value, or adding a value to all element in the subsegment).

In general, a Segment Tree is a very flexible data structure, and a huge number of problems can be solved with it. Additionally, it is also possible to apply more complex operations and answer more complex queries (see [Advanced versions of Segment Trees](#)). In particular the Segment Tree can be easily generalized to larger dimensions. For instance, with a two-dimensional Segment Tree you can answer sum or minimum queries over some subrectangle of a given matrix in only $O(\log^2 n)$ time.

One important property of Segment Trees is that they require only a linear amount of memory. The standard Segment Tree requires $4n$ vertices for working on an array of size n .

Simplest form of a Segment Tree

To start easy, we consider the simplest form of a Segment Tree. We want to answer sum queries efficiently. The formal definition of our task is: Given an array $a[0 \dots n - 1]$, the Segment Tree must be able to find the sum of elements between the indices l and r (i.e. computing the sum $\sum_{i=l}^r a[i]$), and also handle changing values of the elements in the array (i.e. perform assignments of the form $a[i] = x$). The Segment Tree should be able to process **both** queries in $O(\log n)$ time.

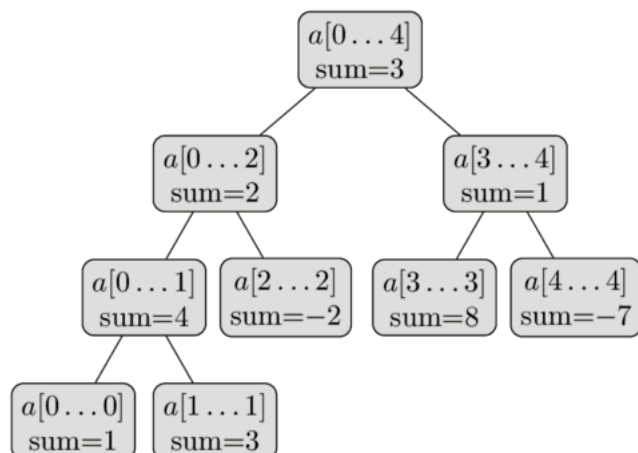
This is an improvement over the simpler approaches. A naive array implementation - just using a simple array - can update elements in $O(1)$, but requires $O(n)$ to compute each sum query. And precomputed prefix sums can compute sum queries in $O(1)$, but updating an array element requires $O(n)$ changes to the prefix sums.

Structure of the Segment Tree

We can take a divide-and-conquer approach when it comes to array segments. We compute and store the sum of the elements of the whole array, i.e. the sum of the segment $a[0 \dots n - 1]$. We then split the array into two halves $a[0 \dots n/2 - 1]$ and $a[n/2 \dots n - 1]$ and compute the sum of each half and store them. Each of these two halves in turn are split in half, and so on until all segments reach size 1.

We can view these segments as forming a binary tree: the root of this tree is the segment $a[0 \dots n - 1]$, and each vertex (except leaf vertices) has exactly two child vertices. This is why the data structure is called "Segment Tree", even though in most implementations the tree is not constructed explicitly (see [Implementation](#)).

Here is a visual representation of such a Segment Tree over the array $a = [1, 3, -2, 8, -7]$:



From this short description of the data structure, we can already conclude that a Segment Tree only requires a linear number of vertices. The first level of the tree contains a single node (the root), the second level will contain two vertices, in the third it will contain four vertices, until the number of vertices reaches n . Thus the number of vertices in the worst case can be estimated by the sum

$$1 + 2 + 4 + \dots + 2^{\lceil \log_2 n \rceil} < 2^{\lceil \log_2 n \rceil + 1} < 4n.$$

It is worth noting that whenever n is not a power of two, not all levels of the Segment Tree will be completely filled. We can see that behavior in the image. For now we can forget about this fact, but it will become important later during the implementation.

The height of the Segment Tree is $O(\log n)$, because when going down from the root to the leaves the size of the segments decreases approximately by half.

Construction

Before constructing the segment tree, we need to decide:

1. the *value* that gets stored at each node of the segment tree. For example, in a sum segment tree, a node would store the sum of the elements in its range $[l, r]$.
2. the *merge* operation that merges two siblings in a segment tree. For example, in a sum segment tree, the two nodes corresponding to the ranges $a[l_1 \dots r_1]$ and $a[l_2 \dots r_2]$ would be merged into a node corresponding to the range $a[l_1 \dots r_2]$ by adding the values of the two nodes.

Note that a vertex is a "leaf vertex", if its corresponding segment covers only one value in the original array. It is present at the lowermost level of a segment tree. Its value would be equal to the (corresponding) element $a[i]$.

Now, for construction of the segment tree, we start at the bottom level (the leaf vertices) and assign them their respective values. On the basis of these values, we can compute the values of the previous level, using the `merge` function. And on the basis of those, we can compute the values of the previous, and repeat the procedure until we reach the root vertex.

It is convenient to describe this operation recursively in the other direction, i.e., from the root vertex to the leaf vertices. The construction procedure, if called on a non-leaf vertex, does the following:

1. recursively construct the values of the two child vertices
2. merge the computed values of these children.

We start the construction at the root vertex, and hence, we are able to compute the entire segment tree.

The time complexity of this construction is $O(n)$, assuming that the merge operation is constant time (the merge operation gets called n times, which is equal to the number of internal nodes in the segment tree).

Sum queries

For now we are going to answer sum queries. As an input we receive two integers l and r , and we have to compute the sum of the segment $a[l \dots r]$ in $O(\log n)$ time.

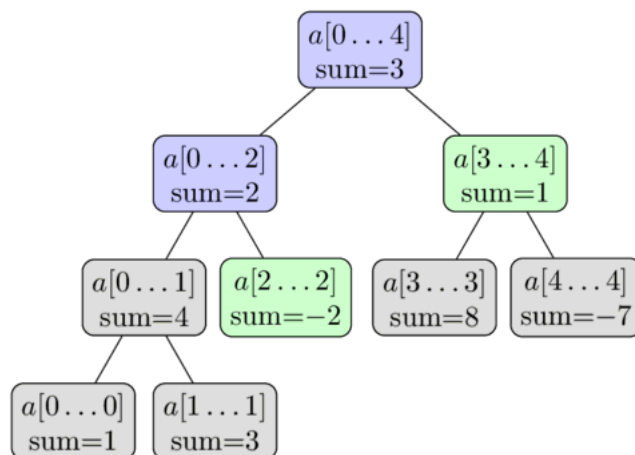
To do this, we will traverse the Segment Tree and use the precomputed sums of the segments. Let's assume that we are currently at the vertex that covers the segment $a[t_l \dots t_r]$. There are three possible cases: 1. $[t_l \dots t_r]$ is completely contained in $[l \dots r]$: In this case, we know that the sum of this segment will surely be a part of the final answer, so we return this sum. 2. $[t_l \dots t_r]$ partially covers $[l \dots r]$: In this case, we cannot include this vertex's sum and we must explore both the left and right children. First we go to the left child($[t_l \dots t_m]$), compute a partial answer for this vertex (i.e. the sum of values of the intersection between the segment of the query and the segment of the left child), then go to the right child($[t_m + 1 \dots t_r]$), compute the partial answer using that vertex, and then combine the answers by adding them. 3. There is no intersection between $[t_l \dots t_r]$ and $[l \dots r]$: This segment does not contribute to the sum, so we return zero.

So, processing a sum query is a function that recursively calls itself with left and right children until it finds a complete or no overlap between ranges.

In other words, the calculation of the query is a traversal of the tree, which spreads through all necessary branches of the tree, and uses the precomputed sum values of the segments in the tree.

Obviously we will start the traversal from the root vertex of the Segment Tree.

The procedure is illustrated in the following image. Again the array $a = [1, 3, -2, 8, -7]$ is used, and here we want to compute the sum $\sum_{i=2}^4 a[i]$. The colored vertices will be visited, and we will use the precomputed values of the green vertices. This gives us the result $-2 + 1 = -1$.



Why is the complexity of this algorithm $O(\log n)$? To show this complexity we look at each level of the tree. It turns out, that for each level we only visit not more than four vertices. And since the height of the tree is $O(\log n)$, we receive the desired running time.

We can show that this proposition (at most four vertices each level) is true by induction. At the first level, we only visit one vertex, the root vertex, so here we visit less than four vertices. Now let's look at an arbitrary level. By induction hypothesis, we visit at most four vertices. If we only visit at most two vertices, the next level has at most four vertices. That is trivial, because each vertex can only cause at most two recursive calls. So let's assume that we visit three or four vertices in the current level. From those vertices, we will analyze the vertices in the middle more carefully. Since the sum query asks for the sum of a continuous subarray, we know that segments corresponding to the visited vertices in the middle will be completely covered by the segment of the sum query. Therefore these vertices will not make any recursive calls. So only the most left, and the most right vertex will have the potential to make recursive calls. And those will only create at most four recursive calls, so also the next level will satisfy the assertion. We can say that one branch approaches the left boundary of the query, and the second branch approaches the right one.

Therefore we visit at most $4 \log n$ vertices in total, and that is equal to a running time of $O(\log n)$.

In conclusion the query works by dividing the input segment into several sub-segments for which all the sums are already precomputed and stored in the tree. And if we stop partitioning whenever the query segment coincides with the vertex segment, then we only need $O(\log n)$ such segments, which gives the effectiveness of the Segment Tree.

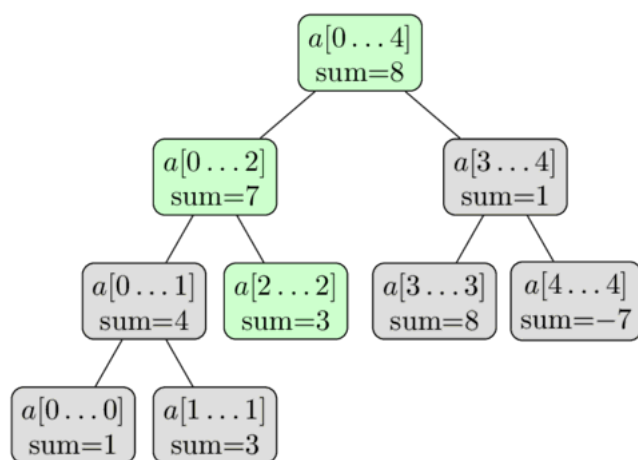
Update queries

Now we want to modify a specific element in the array, let's say we want to do the assignment $a[i] = x$. And we have to rebuild the Segment Tree, such that it corresponds to the new, modified array.

This query is easier than the sum query. Each level of a Segment Tree forms a partition of the array. Therefore an element $a[i]$ only contributes to one segment from each level. Thus only $O(\log n)$ vertices need to be updated.

It is easy to see, that the update request can be implemented using a recursive function. The function gets passed the current tree vertex, and it recursively calls itself with one of the two child vertices (the one that contains $a[i]$ in its segment), and after that recomputes its sum value, similar how it is done in the build method (that is as the sum of its two children).

Again here is a visualization using the same array. Here we perform the update $a[2] = 3$. The green vertices are the vertices that we visit and update.



Implementation

The main consideration is how to store the Segment Tree. Of course we can define a `Vertex` struct and create objects, that store the boundaries of the segment, its sum and additionally also pointers to its child vertices. However, this requires storing a lot of redundant information in the form of pointers. We will use a simple trick to make this a lot more efficient by using an *implicit data structure*: Only storing the sums in an array. (A similar method is used for binary heaps). The sum of the root vertex at index 1, the sums of its two child vertices at indices 2 and 3, the sums of the children of those two vertices at indices 4 to 7, and so on. With 1-indexing, conveniently the left child of a vertex at index i is stored at index $2i$, and the right one at index $2i + 1$. Equivalently, the parent of a vertex at index i is stored at $i / 2$ (integer division).

This simplifies the implementation a lot. We don't need to store the structure of the tree in memory. It is defined implicitly. We only need one array which contains the sums of all segments.

As noted before, we need to store at most $4n$ vertices. It might be less, but for convenience we always allocate an array of size $4n$. There will be some elements in the sum array, that will not correspond to any vertices in the actual tree, but this doesn't complicate the implementation.

So, we store the Segment Tree simply as an array `t[]` with a size of four times the input size n :

```
int n, t[4*MAXN];
```

The procedure for constructing the Segment Tree from a given array `a[]` looks like this: it is a recursive function with the parameters `a[]` (the input array), `v` (the index of the current vertex), and the boundaries `tl` and `tr` of the current segment. In the main program this function will be called with the parameters of the root vertex: `v = 1`, `tl = 0`, and `tr = n - 1`.

```
void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = a[tl];
    } else {
        int tm = (tl + tr) / 2;
```

```

        build(a, v*2, t1, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = t[v*2] + t[v*2+1];
    }
}

```

Further the function for answering sum queries is also a recursive function, which receives as parameters information about the current vertex/segment (i.e. the index v and the boundaries $t1$ and tr) and also the information about the boundaries of the query, l and r . In order to simplify the code, this function always does two recursive calls, even if only one is necessary - in that case the superfluous recursive call will have $l > r$, and this can easily be caught using an additional check at the beginning of the function.

```

int sum(int v, int t1, int tr, int l, int r) {
    if (tr < l || t1 > r) return 0; // no overlap
    if (l <= t1 && tr <= r) return t[v]; // nested segment

    int tm = (t1 + tr) / 2;
    // partial overlap
    return sum(v * 2, t1, tm, l, r)
        + sum(v * 2 + 1, tm + 1, tr, l, r);
}

```

Finally the update query. The function will also receive information about the current vertex/segment, and additionally also the parameter of the update query (i.e. the position of the element and its new value).

```

void update(int v, int t1, int tr, int pos, int new_val) {
    if (t1 == tr) {
        t[v] = new_val;
    } else {
        int tm = (t1 + tr) / 2;
        if (pos <= tm)
            update(v*2, t1, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = t[v*2] + t[v*2+1];
    }
}

```

Memory efficient implementation

Most people use the implementation from the previous section. If you look at the array `t` you can see that it follows the numbering of the tree nodes in the order of a BFS traversal (level-order traversal). Using this traversal the children of vertex v are $2v$ and $2v + 1$ respectively. However if n is not a power of two, this method will skip some indices and leave some parts of the array `t` unused. The memory consumption is limited by $4n$, even though a Segment Tree of an array of n elements requires only $2n - 1$ vertices.

However it can be reduced. We renumber the vertices of the tree in the order of an Euler tour traversal (pre-order traversal), and we write all these vertices next to each other.

Let's look at a vertex at index v , and let it be responsible for the segment $[l, r]$, and let $mid = \frac{l+r}{2}$. It is obvious that the left child will have the index $v + 1$. The left child is responsible for the segment $[l, mid]$, i.e. in total there will be $2 * (mid - l + 1) - 1$ vertices in the left child's subtree. Thus we can compute the index of the right child of v . The index will be $v + 2 * (mid - l + 1)$. By this numbering we achieve a reduction of the necessary memory to $2n$.

Advanced versions of Segment Trees

A Segment Tree is a very flexible data structure, and allows variations and extensions in many different directions. Let's try to categorize them below.

More complex queries

It can be quite easy to change the Segment Tree in a direction, such that it computes different queries (e.g. computing the minimum / maximum instead of the sum), but it also can be very nontrivial.

Finding the maximum

Let us slightly change the condition of the problem described above: instead of querying the sum, we will now make maximum queries.

The tree will have exactly the same structure as the tree described above. We only need to change the way $t[v]$ is computed in the `build` and `update` functions. $t[v]$ will now store the maximum of the corresponding segment. And we also need to change the calculation of the returned value of the `sum` function (replacing the summation by the maximum).

Of course this problem can be easily changed into computing the minimum instead of the maximum.

Instead of showing an implementation to this problem, the implementation will be given to a more complex version of this problem in the next section.

Finding the maximum and the number of times it appears

This task is very similar to the previous one. In addition of finding the maximum, we also have to find the number of occurrences of the maximum.

To solve this problem, we store a pair of numbers at each vertex in the tree: In addition to the maximum we also store the number of occurrences of it in the corresponding segment. Determining the correct pair to store at $t[v]$ can still be done in constant time using the information of the pairs stored at the child vertices. Combining two such pairs should be done in a separate function, since this will be an operation that we will do while building the tree, while answering maximum queries and while performing modifications.

```
pair<int, int> t[4*MAXN];

pair<int, int> combine(pair<int, int> a, pair<int, int> b) {
    if (a.first > b.first)
        return a;
    if (b.first > a.first)
        return b;
    return make_pair(a.first, a.second + b.second);
}

void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = make_pair(a[tl], 1);
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

pair<int, int> get_max(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return make_pair(-INF, 0);
    if (l == tl && r == tr)
        return t[v];
    int tm = (tl + tr) / 2;
    return combine(get_max(v*2, tl, tm, l, min(r, tm)),
                  get_max(v*2+1, tm+1, tr, max(l, tm+1), r));
}

void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
        t[v] = make_pair(new_val, 1);
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}
```

Compute the greatest common divisor / least common multiple

In this problem we want to compute the GCD / LCM of all numbers in given ranges of the array.

This interesting variation of the Segment Tree can be solved in exactly the same way as the Segment Trees we derived for sum / minimum / maximum queries: it is enough to store the GCD / LCM of the corresponding vertex in each vertex of the tree. Combining two vertices can be done by computing the GCD / LCM of both vertices.

Counting the number of zeros, searching for the k-th zero

In this problem we want to find the number of zeros in a given range, and additionally find the index of the k-th zero using a second function.

Again we have to change the store values of the tree a bit: This time we will store the number of zeros in each segment in $t[]$. It is pretty clear, how to implement the `build`, `update` and `count_zero` functions, we can simply use the ideas from the sum query problem. Thus we solved the first part of the problem.

Now we learn how to solve the problem of finding the k-th zero in the array $a[]$. To do this task, we will descend the Segment Tree, starting at the root vertex, and moving each time to either the left or the right child, depending on which segment contains the k-th zero. In order to decide to which child we need to go, it is enough to look at the number of zeros appearing in the segment corresponding to the left vertex. If this precomputed count is greater or equal to k, it is necessary to descend to the left child, and otherwise descent to the right child. Notice, if we chose the right child, we have to subtract the number of zeros of the left child from k.

In the implementation we can handle the special case, $a[]$ containing less than k zeros, by returning -1.

```
int find_kth(int v, int tl, int tr, int k) {
    if (k > t[v])
        return -1;
    if (tl == tr)
        return tl;
    int tm = (tl + tr) / 2;
    if (t[v*2] >= k)
        return find_kth(v*2, tl, tm, k);
    else
        return find_kth(v*2+1, tm+1, tr, k - t[v*2]);
}
```

Searching for an array prefix with a given amount

The task is as follows: for a given value x we have to quickly find smallest index i such that the sum of the first i elements of the array $a[]$ is greater or equal to x (assuming that the array $a[]$ only contains non-negative values).

This task can be solved using binary search, computing the sum of the prefixes with the Segment Tree. However this will lead to a $O(\log^2 n)$ solution.

Instead we can use the same idea as in the previous section, and find the position by descending the tree: by moving each time to the left or the right, depending on the sum of the left child. Thus finding the answer in $O(\log n)$ time.

Searching for the first element greater than a given amount

The task is as follows: for a given value x and a range $a[l \dots r]$ find the smallest i in the range $a[l \dots r]$, such that $a[i]$ is greater than x.

This task can be solved using binary search over max prefix queries with the Segment Tree. However, this will lead to a $O(\log^2 n)$ solution.

Instead, we can use the same idea as in the previous sections, and find the position by descending the tree: by moving each time to the left or the right, depending on the maximum value of the left child. Thus finding the answer in $O(\log n)$ time.

```
int get_first(int v, int tl, int tr, int l, int r, int x) {
    if (tl > r || tr < l) return -1;
    if (t[v] <= x) return -1;

    if (tl == tr) return tl;

    int tm = tl + (tr - tl) / 2;
    int left = get_first(2*v, tl, tm, l, r, x);
    if (left != -1) return left;
    return get_first(2*v+1, tm+1, tr, l, r, x);
}
```

Finding subsegments with the maximal sum

Here again we receive a range $a[l \dots r]$ for each query, this time we have to find a subsegment $a[l' \dots r']$ such that $l \leq l'$ and $r' \leq r$ and the sum of the elements of this segment is maximal. As before we also want to be able to modify individual elements of the array. The elements of the array can be negative, and the optimal subsegment can be empty (e.g. if all elements are negative).

This problem is a non-trivial usage of a Segment Tree. This time we will store four values for each vertex: the sum of the segment, the maximum prefix sum, the maximum suffix sum, and the sum of the maximal subsegment in it. In other words for each segment of the Segment Tree the answer is already precomputed as well as the answers for segments touching the left and the right boundaries of the segment.

How to build a tree with such data? Again we compute it in a recursive fashion: we first compute all four values for the left and the right child, and then combine those to archive the four values for the current vertex. Note the answer for the current vertex is either:

- the answer of the left child, which means that the optimal subsegment is entirely placed in the segment of the left child
- the answer of the right child, which means that the optimal subsegment is entirely placed in the segment of the right child
- the sum of the maximum suffix sum of the left child and the maximum prefix sum of the right child, which means that the optimal subsegment intersects with both children.

Hence the answer to the current vertex is the maximum of these three values. Computing the maximum prefix / suffix sum is even easier. Here is the implementation of the combine function, which receives only data from the left and right child, and returns the data of the current vertex.

```
struct data {
    int sum, pref, suff, ans;
};

data combine(data l, data r) {
    data res;
    res.sum = l.sum + r.sum;
    res.pref = max(l.pref, l.sum + r.pref);
    res.suff = max(r.suff, r.sum + l.suff);
    res.ans = max(max(l.ans, r.ans), l.suff + r.pref);
    return res;
}
```

Using the combine function it is easy to build the Segment Tree. We can implement it in exactly the same way as in the previous implementations. To initialize the leaf vertices, we additionally create the auxiliary function make_data, which will return a data object holding the information of a single value.

```
data make_data(int val) {
    data res;
    res.sum = val;
    res.pref = res.suff = res.ans = max(0, val);
    return res;
}

void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = make_data(a[tl]);
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
        t[v] = make_data(new_val);
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}
```

It only remains, how to compute the answer to a query. To answer it, we go down the tree as before, breaking the query into several subsegments that coincide with the segments of the Segment Tree, and combine the answers in them into a single answer for the query. Then it should be clear, that the work is exactly the same as in the simple Segment Tree, but instead of summing / minimizing / maximizing the values, we use the combine function.

```
data query(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return make_data(0);
```

```

    if (l == tl && r == tr)
        return t[v];
    int tm = (tl + tr) / 2;
    return combine(query(v*2, tl, tm, l, min(r, tm)),
                  query(v*2+1, tm+1, tr, max(l, tm+1), r));
}

```

Saving the entire subarrays in each vertex

This is a separate subsection that stands apart from the others, because at each vertex of the Segment Tree we don't store information about the corresponding segment in compressed form (sum, minimum, maximum, ...), but store all elements of the segment. Thus the root of the Segment Tree will store all elements of the array, the left child vertex will store the first half of the array, the right vertex the second half, and so on.

In its simplest application of this technique we store the elements in sorted order. In more complex versions the elements are not stored in lists, but more advanced data structures (sets, maps, ...). But all these methods have the common factor, that each vertex requires linear memory (i.e. proportional to the length of the corresponding segment).

The first natural question, when considering these Segment Trees, is about memory consumption. Intuitively this might look like $O(n^2)$ memory, but it turns out that the complete tree will only need $O(n \log n)$ memory. Why is this so? Quite simply, because each element of the array falls into $O(\log n)$ segments (remember the height of the tree is $O(\log n)$).

So in spite of the apparent extravagance of such a Segment Tree, it consumes only slightly more memory than the usual Segment Tree.

Several typical applications of this data structure are described below. It is worth noting the similarity of these Segment Trees with 2D data structures (in fact this is a 2D data structure, but with rather limited capabilities).

Find the smallest number greater or equal to a specified number. No modification queries.

We want to answer queries of the following form: for three given numbers (l, r, x) we have to find the minimal number in the segment $a[l \dots r]$ which is greater than or equal to x .

We construct a Segment Tree. In each vertex we store a sorted list of all numbers occurring in the corresponding segment, like described above. How to build such a Segment Tree as effectively as possible? As always we approach this problem recursively: let the lists of the left and right children already be constructed, and we want to build the list for the current vertex. From this view the operation is now trivial and can be accomplished in linear time: We only need to combine the two sorted lists into one, which can be done by iterating over them using two pointers. The C++ STL already has an implementation of this algorithm.

Because this structure of the Segment Tree and the similarities to the merge sort algorithm, the data structure is also often called "Merge Sort Tree".

```

vector<int> t[4*MAXN];

void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = vector<int>(1, a[tl]);
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        merge(t[v*2].begin(), t[v*2].end(), t[v*2+1].begin(), t[v*2+1].end(),
              back_inserter(t[v]));
    }
}

```

We already know that the Segment Tree constructed in this way will require $O(n \log n)$ memory. And thanks to this implementation its construction also takes $O(n \log n)$ time, after all each list is constructed in linear time in respect to its size.

Now consider the answer to the query. We will go down the tree, like in the regular Segment Tree, breaking our segment $a[l \dots r]$ into several subsegments (into at most $O(\log n)$ pieces). It is clear that the answer of the whole answer is the minimum of each of the subqueries. So now we only need to understand, how to respond to a query on one such subsegment that corresponds with some vertex of the tree.

We are at some vertex of the Segment Tree and we want to compute the answer to the query, i.e. find the minimum number greater than or equal to a given number x . Since the vertex contains the list of elements in sorted order, we can simply perform a binary search on this list and return the first number, greater than or equal to x .

Thus the answer to the query in one segment of the tree takes $O(\log n)$ time, and the entire query is processed in $O(\log^2 n)$.

```
int query(int v, int tl, int tr, int l, int r, int x) {
    if (l > r)
        return INF;
    if (l == tl && r == tr) {
        vector<int>::iterator pos = lower_bound(t[v].begin(), t[v].end(), x);
        if (pos != t[v].end())
            return *pos;
        return INF;
    }
    int tm = (tl + tr) / 2;
    return min(query(v*2, tl, tm, l, min(r, tm), x),
               query(v*2+1, tm+1, tr, max(l, tm+1), r, x));
}
```

The constant `INF` is equal to some large number that is bigger than all numbers in the array. Its usage means, that there is no number greater than or equal to `x` in the segment. It has the meaning of "there is no answer in the given interval".

Find the smallest number greater or equal to a specified number. With modification queries.

This task is similar to the previous. The last approach has a disadvantage, it was not possible to modify the array between answering queries. Now we want to do exactly this: a modification query will do the assignment $a[i] = y$.

The solution is similar to the solution of the previous problem, but instead of lists at each vertex of the Segment Tree, we will store a balanced list that allows you to quickly search for numbers, delete numbers, and insert new numbers. Since the array can contain a number repeated, the optimal choice is the data structure `multiset`.

The construction of such a Segment Tree is done in pretty much the same way as in the previous problem, only now we need to combine `multiset`s and not sorted lists. This leads to a construction time of $O(n \log^2 n)$ (in general merging two red-black trees can be done in linear time, but the C++ STL doesn't guarantee this time complexity).

The query function is also almost equivalent, only now the `lower_bound` function of the `multiset` function should be called instead (`std::lower_bound` only works in $O(\log n)$ time if used with random-access iterators).

Finally the modification request. To process it, we must go down the tree, and modify all `multiset` from the corresponding segments that contain the affected element. We simply delete the old value of this element (but only one occurrence), and insert the new value.

```
void update(int v, int tl, int tr, int pos, int new_val) {
    t[v].erase(t[v].find(a[pos]));
    t[v].insert(new_val);
    if (tl != tr) {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
    } else {
        a[pos] = new_val;
    }
}
```

Processing of this modification query also takes $O(\log^2 n)$ time.

Find the smallest number greater or equal to a specified number. Acceleration with "fractional cascading".

We have the same problem statement, we want to find the minimal number greater than or equal to `x` in a segment, but this time in $O(\log n)$ time. We will improve the time complexity using the technique "fractional cascading".

Fractional cascading is a simple technique that allows you to improve the running time of multiple binary searches, which are conducted at the same time. Our previous approach to the search query was, that we divide the task into several subtasks, each of which is solved with a binary search. Fractional cascading allows you to replace all of these binary searches with a single one.

The simplest and most obvious example of fractional cascading is the following problem: there are k sorted lists of numbers, and we must find in each list the first number greater than or equal to the given number.

Instead of performing a binary search for each list, we could merge all lists into one big sorted list. Additionally for each element `y` we store a list of results of searching for `y` in each of the k lists. Therefore if we want to find the smallest number greater than or equal to `x`, we just need

to perform one single binary search, and from the list of indices we can determine the smallest number in each list. This approach however requires $O(n \cdot k)$ (n is the length of the combined lists), which can be quite inefficient.

Fractional cascading reduces this memory complexity to $O(n)$ memory, by creating from the k input lists k new lists, in which each list contains the corresponding list and additionally also every second element of the following new list. Using this structure it is only necessary to store two indices, the index of the element in the original list, and the index of the element in the following new list. So this approach only uses $O(n)$ memory, and still can answer the queries using a single binary search.

But for our application we do not need the full power of fractional cascading. In our Segment Tree a vertex will contain the sorted list of all elements that occur in either the left or the right subtrees (like in the Merge Sort Tree). Additionally to this sorted list, we store two positions for each element. For an element y we store the smallest index i , such that the i th element in the sorted list of the left child is greater or equal to y . And we store the smallest index j , such that the j th element in the sorted list of the right child is greater or equal to y . These values can be computed in parallel to the merging step when we build the tree.

How does this speed up the queries?

Remember, in the normal solution we did a binary search in every node. But with this modification, we can avoid all except one.

To answer a query, we simply do a binary search in the root node. This gives us the smallest element $y \geq x$ in the complete array, but it also gives us two positions. The index of the smallest element greater or equal x in the left subtree, and the index of the smallest element y in the right subtree. Notice that $\geq y$ is the same as $\geq x$, since our array doesn't contain any elements between x and y . In the normal Merge Sort Tree solution we would compute these indices via binary search, but with the help of the precomputed values we can just look them up in $O(1)$. And we can repeat that until we visited all nodes that cover our query interval.

To summarize, as usual we touch $O(\log n)$ nodes during a query. In the root node we do a binary search, and in all other nodes we only do constant work. This means the complexity for answering a query is $O(\log n)$.

But notice, that this uses three times more memory than a normal Merge Sort Tree, which already uses a lot of memory ($O(n \log n)$).

It is straightforward to apply this technique to a problem, that doesn't require any modification queries. The two positions are just integers and can easily be computed by counting when merging the two sorted sequences.

It is still possible to also allow modification queries, but that complicates the entire code. Instead of integers, you need to store the sorted array as `multiset`, and instead of indices you need to store iterators. And you need to work very carefully, so that you increment or decrement the correct iterators during a modification query.

Other possible variations

This technique implies a whole new class of possible applications. Instead of storing a vector or a multiset in each vertex, other data structures can be used: other Segment Trees (somewhat discussed in [Generalization to higher dimensions](#)), Fenwick Trees, Cartesian trees, etc.

Range updates (Lazy Propagation)

All problems in the above sections discussed modification queries that only affected a single element of the array each. However the Segment Tree allows applying modification queries to an entire segment of contiguous elements, and perform the query in the same time $O(\log n)$.

Addition on segments

We begin by considering problems of the simplest form: the modification query should add a number x to all numbers in the segment $a[l \dots r]$. The second query, that we are supposed to answer, asked simply for the value of $a[i]$.

To make the addition query efficient, we store at each vertex in the Segment Tree how many we should add to all numbers in the corresponding segment. For example, if the query "add 3 to the whole array $a[0 \dots n - 1]$ " comes, then we place the number 3 in the root of the tree. In general we have to place this number to multiple segments, which form a partition of the query segment. Thus we don't have to change all $O(n)$ values, but only $O(\log n)$ many.

If now there comes a query that asks the current value of a particular array entry, it is enough to go down the tree and add up all values found along the way.

```
void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = a[tl];
    } else {
        int tm = (tl + tr) / 2;
```

```

        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = 0;
    }
}

void update(int v, int tl, int tr, int l, int r, int add) {
    if (l > r)
        return;
    if (l == tl && r == tr) {
        t[v] += add;
    } else {
        int tm = (tl + tr) / 2;
        update(v*2, tl, tm, l, min(r, tm), add);
        update(v*2+1, tm+1, tr, max(l, tm+1), r, add);
    }
}

int get(int v, int tl, int tr, int pos) {
    if (tl == tr)
        return t[v];
    int tm = (tl + tr) / 2;
    if (pos <= tm)
        return t[v] + get(v*2, tl, tm, pos);
    else
        return t[v] + get(v*2+1, tm+1, tr, pos);
}

```

Assignment on segments

Suppose now that the modification query asks to assign each element of a certain segment $a[l \dots r]$ to some value p . As a second query we will again consider reading the value of the array $a[i]$.

To perform this modification query on a whole segment, you have to store at each vertex of the Segment Tree whether the corresponding segment is covered entirely with the same value or not. This allows us to make a "lazy" update: instead of changing all segments in the tree that cover the query segment, we only change some, and leave others unchanged. A marked vertex will mean, that every element of the corresponding segment is assigned to that value, and actually also the complete subtree should only contain this value. In a sense we are lazy and delay writing the new value to all those vertices. We can do this tedious task later, if this is necessary.

So after the modification query is executed, some parts of the tree become irrelevant - some modifications remain unfulfilled in it.

For example if a modification query "assign a number to the whole array $a[0 \dots n - 1]$ " gets executed, in the Segment Tree only a single change is made - the number is placed in the root of the tree and this vertex gets marked. The remaining segments remain unchanged, although in fact the number should be placed in the whole tree.

Suppose now that the second modification query says, that the first half of the array $a[0 \dots n/2]$ should be assigned with some other number. To process this query we must assign each element in the whole left child of the root vertex with that number. But before we do this, we must first sort out the root vertex first. The subtlety here is that the right half of the array should still be assigned to the value of the first query, and at the moment there is no information for the right half stored.

The way to solve this is to push the information of the root to its children, i.e. if the root of the tree was assigned with any number, then we assign the left and the right child vertices with this number and remove the mark of the root. After that, we can assign the left child with the new value, without losing any necessary information.

Summarizing we get: for any queries (a modification or reading query) during the descent along the tree we should always push information from the current vertex into both of its children. We can understand this in such a way, that when we descent the tree we apply delayed modifications, but exactly as much as necessary (so not to degrade the complexity of $O(\log n)$).

For the implementation we need to make a `push` function, which will receive the current vertex, and it will push the information for its vertex to both its children. We will call this function at the beginning of the query functions (but we will not call it from the leaves, because there is no need to push information from them any further).

```

void push(int v) {
    if (marked[v]) {
        t[v*2] = t[v*2+1] = t[v];
        marked[v*2] = marked[v*2+1] = true;
        marked[v] = false;
    }
}

void update(int v, int tl, int tr, int l, int r, int new_val) {

```

```

    if (l > r)
        return;
    if (l == t1 && tr == r) {
        t[v] = new_val;
        marked[v] = true;
    } else {
        push(v);
        int tm = (t1 + tr) / 2;
        update(v*2, t1, tm, l, min(r, tm), new_val);
        update(v*2+1, tm+1, tr, max(l, tm+1), r, new_val);
    }
}

int get(int v, int t1, int tr, int pos) {
    if (t1 == tr) {
        return t[v];
    }
    push(v);
    int tm = (t1 + tr) / 2;
    if (pos <= tm)
        return get(v*2, t1, tm, pos);
    else
        return get(v*2+1, tm+1, tr, pos);
}

```

Notice: the function `get` can also be implemented in a different way: do not make delayed updates, but immediately return the value `t[v]` if `marked[v]` is true.

Adding on segments, querying for maximum

Now the modification query is to add a number to all elements in a range, and the reading query is to find the maximum in a range.

So for each vertex of the Segment Tree we have to store the maximum of the corresponding subsegment. The interesting part is how to recompute these values during a modification request.

For this purpose we keep store an additional value for each vertex. In this value we store the addends we haven't propagated to the child vertices. Before traversing to a child vertex, we call `push` and propagate the value to both children. We have to do this in both the `update` function and the `query` function.

```

void build(int a[], int v, int t1, int tr) {
    if (t1 == tr) {
        t[v] = a[t1];
    } else {
        int tm = (t1 + tr) / 2;
        build(a, v*2, t1, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = max(t[v*2], t[v*2+1]);
    }
}

void push(int v) {
    t[v*2] += lazy[v];
    lazy[v*2] += lazy[v];
    t[v*2+1] += lazy[v];
    lazy[v*2+1] += lazy[v];
    lazy[v] = 0;
}

void update(int v, int t1, int tr, int l, int r, int addend) {
    if (l > r)
        return;
    if (l == t1 && tr == r) {
        t[v] += addend;
        lazy[v] += addend;
    } else {
        push(v);
        int tm = (t1 + tr) / 2;
        update(v*2, t1, tm, l, min(r, tm), addend);
        update(v*2+1, tm+1, tr, max(l, tm+1), r, addend);
        t[v] = max(t[v*2], t[v*2+1]);
    }
}

int query(int v, int t1, int tr, int l, int r) {
    if (l > r)
        return -INF;
}

```

```

    if (l == tl && tr == r)
        return t[v];
    push(v);
    int tm = (tl + tr) / 2;
    return max(query(v*2, tl, tm, l, min(r, tm)),
               query(v*2+1, tm+1, tr, max(l, tm+1), r));
}

```

Generalization to higher dimensions

A Segment Tree can be generalized quite natural to higher dimensions. If in the one-dimensional case we split the indices of the array into segments, then in the two-dimensional we make an ordinary Segment Tree with respect to the first indices, and for each segment we build an ordinary Segment Tree with respect to the second indices.

Simple 2D Segment Tree

A matrix $a[0 \dots n - 1, 0 \dots m - 1]$ is given, and we have to find the sum (or minimum/maximum) on some submatrix $a[x_1 \dots x_2, y_1 \dots y_2]$, as well as perform modifications of individual matrix elements (i.e. queries of the form $a[x][y] = p$).

So we build a 2D Segment Tree: first the Segment Tree using the first coordinate (x), then the second (y).

To make the construction process more understandable, you can forget for a while that the matrix is two-dimensional, and only leave the first coordinate. We will construct an ordinary one-dimensional Segment Tree using only the first coordinate. But instead of storing a number in a segment, we store an entire Segment Tree: i.e. at this moment we remember that we also have a second coordinate; but because at this moment the first coordinate is already fixed to some interval $[l \dots r]$, we actually work with such a strip $a[l \dots r, 0 \dots m - 1]$ and for it we build a Segment Tree.

Here is the implementation of the construction of a 2D Segment Tree. It actually represents two separate blocks: the construction of a Segment Tree along the x coordinate (build_x), and the y coordinate (build_y). For the leaf nodes in build_y we have to separate two cases: when the current segment of the first coordinate $[tlx \dots trx]$ has length 1, and when it has a length greater than one. In the first case, we just take the corresponding value from the matrix, and in the second case we can combine the values of two Segment Trees from the left and the right son in the coordinate x .

```

void build_y(int vx, int lx, int rx, int vy, int ly, int ry) {
    if (ly == ry) {
        if (lx == rx)
            t[vx][vy] = a[lx][ly];
        else
            t[vx][vy] = t[vx*2][vy] + t[vx*2+1][vy];
    } else {
        int my = (ly + ry) / 2;
        build_y(vx, lx, rx, vy*2, ly, my);
        build_y(vx, lx, rx, vy*2+1, my+1, ry);
        t[vx][vy] = t[vx][vy*2] + t[vx][vy*2+1];
    }
}

void build_x(int vx, int lx, int rx) {
    if (lx != rx) {
        int mx = (lx + rx) / 2;
        build_x(vx*2, lx, mx);
        build_x(vx*2+1, mx+1, rx);
    }
    build_y(vx, lx, rx, 1, 0, m-1);
}

```

Such a Segment Tree still uses a linear amount of memory, but with a larger constant: $16nm$. It is clear that the described procedure build_x also works in linear time.

Now we turn to processing of queries. We will answer to the two-dimensional query using the same principle: first break the query on the first coordinate, and then for every reached vertex, we call the corresponding Segment Tree of the second coordinate.

```

int sum_y(int vx, int vy, int tly, int try_, int ly, int ry) {
    if (ly > ry)
        return 0;
    if (ly == tly && try_ == ry)
        return t[vx][vy];
    int tmy = (tly + try_) / 2;
    return sum_y(vx, vy*2, tly, tmy, ly, min(ry, tmy))
        + sum_y(vx, vy*2+1, tmy+1, try_, max(ly, tmy+1), ry);
}

```

```

}

int sum_x(int vx, int tlx, int trx, int lx, int rx, int ly, int ry) {
    if (lx > rx)
        return 0;
    if (lx == tlx && trx == rx)
        return sum_y(vx, 1, 0, m-1, ly, ry);
    int tmx = (tlx + trx) / 2;
    return sum_x(vx*2, tlx, tmx, lx, min(rx, tmx), ly, ry)
        + sum_x(vx*2+1, tmx+1, trx, max(lx, tmx+1), rx, ly, ry);
}

```

This function works in $O(\log n \log m)$ time, since it first descends the tree in the first coordinate, and for each traversed vertex in the tree it makes a query in the corresponding Segment Tree along the second coordinate.

Finally we consider the modification query. We want to learn how to modify the Segment Tree in accordance with the change in the value of some element $a[x][y] = p$. It is clear, that the changes will occur only in those vertices of the first Segment Tree that cover the coordinate x (and such will be $O(\log n)$), and for Segment Trees corresponding to them the changes will only occurs at those vertices that covers the coordinate y (and such will be $O(\log m)$). Therefore the implementation will be not very different from the one-dimensional case, only now we first descend the first coordinate, and then the second.

```

void update_y(int vx, int lx, int rx, int vy, int ly, int ry, int x, int y, int new_val) {
    if (ly == ry) {
        if (lx == rx)
            t[vx][vy] = new_val;
        else
            t[vx][vy] = t[vx*2][vy] + t[vx*2+1][vy];
    } else {
        int my = (ly + ry) / 2;
        if (y <= my)
            update_y(vx, lx, rx, vy*2, ly, my, x, y, new_val);
        else
            update_y(vx, lx, rx, vy*2+1, my+1, ry, x, y, new_val);
        t[vx][vy] = t[vx][vy*2] + t[vx][vy*2+1];
    }
}

void update_x(int vx, int lx, int rx, int x, int y, int new_val) {
    if (lx != rx) {
        int mx = (lx + rx) / 2;
        if (x <= mx)
            update_x(vx*2, lx, mx, x, y, new_val);
        else
            update_x(vx*2+1, mx+1, rx, x, y, new_val);
    }
    update_y(vx, lx, rx, 1, 0, m-1, x, y, new_val);
}

```

Compression of 2D Segment Tree

Let the problem be the following: there are n points on the plane given by their coordinates (x_i, y_i) and queries of the form "count the number of points lying in the rectangle $((x_1, y_1), (x_2, y_2))$ ". It is clear that in the case of such a problem it becomes unreasonably wasteful to construct a two-dimensional Segment Tree with $O(n^2)$ elements. Most on this memory will be wasted, since each single point can only get into $O(\log n)$ segments of the tree along the first coordinate, and therefore the total "useful" size of all tree segments on the second coordinate is $O(n \log n)$.

So we proceed as follows: at each vertex of the Segment Tree with respect to the first coordinate we store a Segment Tree constructed only by those second coordinates that occur in the current segment of the first coordinates. In other words, when constructing a Segment Tree inside some vertex with index vx and the boundaries tlx and trx , we only consider those points that fall into this interval $x \in [tlx, trx]$, and build a Segment Tree just using them.

Thus we will achieve that each Segment Tree on the second coordinate will occupy exactly as much memory as it should. As a result, the total amount of memory will decrease to $O(n \log n)$. We still can answer the queries in $O(\log^2 n)$ time, we just have to make a binary search on the second coordinate, but this will not worsen the complexity.

But modification queries will be impossible with this structure: in fact if a new point appears, we have to add a new element in the middle of some Segment Tree along the second coordinate, which cannot be effectively done.

In conclusion we note that the two-dimensional Segment Tree constructed in the described way becomes practically equivalent to the modification of the one-dimensional Segment Tree (see [Saving the entire subarrays in each vertex](#)). In particular the two-dimensional Segment Tree is just a special case of storing a subarray in each vertex of the tree. It follows, that if you gave to abandon a two-dimensional Segment Tree due to the impossibility of executing a query, it makes sense to try to replace the nested Segment Tree with some more powerful data structure, for example a Cartesian tree.

Preserving the history of its values (Persistent Segment Tree)

A persistent data structure is a data structure that remembers its previous state for each modification. This allows to access any version of this data structure that interest us and execute a query on it.

Segment Tree is a data structure that can be turned into a persistent data structure efficiently (both in time and memory consumption). We want to avoid copying the complete tree before each modification, and we don't want to lose the $O(\log n)$ time behavior for answering range queries.

In fact, any change request in the Segment Tree leads to a change in the data of only $O(\log n)$ vertices along the path starting from the root. So if we store the Segment Tree using pointers (i.e. a vertex stores pointers to the left and the right child vertices), then when performing the modification query, we simply need to create new vertices instead of changing the available vertices. Vertices that are not affected by the modification query can still be used by pointing the pointers to the old vertices. Thus for a modification query $O(\log n)$ new vertices will be created, including a new root vertex of the Segment Tree, and the entire previous version of the tree rooted at the old root vertex will remain unchanged.

Let's give an example implementation for the simplest Segment Tree: when there is only a query asking for sums, and modification queries of single elements.

```
struct Vertex {
    Vertex *l, *r;
    int sum;

    Vertex(int val) : l(nullptr), r(nullptr), sum(val) {}
    Vertex(Vertex *l, Vertex *r) : l(l), r(r), sum(0) {
        if (l) sum += l->sum;
        if (r) sum += r->sum;
    }
};

Vertex* build(int a[], int tl, int tr) {
    if (tl == tr)
        return new Vertex(a[tl]);
    int tm = (tl + tr) / 2;
    return new Vertex(build(a, tl, tm), build(a, tm+1, tr));
}

int get_sum(Vertex* v, int tl, int tr, int l, int r) {
    if (l > r)
        return 0;
    if (l == tl && tr == r)
        return v->sum;
    int tm = (tl + tr) / 2;
    return get_sum(v->l, tl, tm, l, min(r, tm))
        + get_sum(v->r, tm+1, tr, max(l, tm+1), r);
}

Vertex* update(Vertex* v, int tl, int tr, int pos, int new_val) {
    if (tl == tr)
        return new Vertex(new_val);
    int tm = (tl + tr) / 2;
    if (pos <= tm)
        return new Vertex(update(v->l, tl, tm, pos, new_val), v->r);
    else
        return new Vertex(v->l, update(v->r, tm+1, tr, pos, new_val));
}
```

For each modification of the Segment Tree we will receive a new root vertex. To quickly jump between two different versions of the Segment Tree, we need to store these roots in an array. To use a specific version of the Segment Tree we simply call the query using the appropriate root vertex.

With the approach described above almost any Segment Tree can be turned into a persistent data structure.

Finding the k -th smallest number in a range

This time we have to answer queries of the form "What is the k -th smallest element in the range $a[l \dots r]$ ". This query can be answered using a binary search and a Merge Sort Tree, but the time complexity for a single query would be $O(\log^3 n)$. We will accomplish the same task using a persistent Segment Tree in $O(\log n)$.

First we will discuss a solution for a simpler problem: We will only consider arrays in which the elements are bound by $0 \leq a[i] < n$. And we only want to find the k -th smallest element in some prefix of the array a . It will be very easy to extend the developed ideas later for not restricted arrays and not restricted range queries. Note that we will be using 1 based indexing for a .

We will use a Segment Tree that counts all appearing numbers, i.e. in the Segment Tree we will store the histogram of the array. So the leaf vertices will store how often the values $0, 1, \dots, n-1$ will appear in the array, and the other vertices store how many numbers in some range are in the array. In other words we create a regular Segment Tree with sum queries over the histogram of the array. But instead of creating all n Segment Trees for every possible prefix, we will create one persistent one, that will contain the same information. We will start with an empty Segment Tree (all counts will be 0) pointed to by $root_0$, and add the elements $a[1], a[2], \dots, a[n]$ one after another. For each modification we will receive a new root vertex, let's call $root_i$ the root of the Segment Tree after inserting the first i elements of the array a . The Segment Tree rooted at $root_i$ will contain the histogram of the prefix $a[1 \dots i]$. Using this Segment Tree we can find in $O(\log n)$ time the position of the k -th element using the same technique discussed in [Counting the number of zeros, searching for the \$k\$ -th zero](#).

Now to the not-restricted version of the problem.

First for the restriction on the queries: Instead of only performing these queries over a prefix of a , we want to use any arbitrary segments $a[l \dots r]$. Here we need a Segment Tree that represents the histogram of the elements in the range $a[l \dots r]$. It is easy to see that such a Segment Tree is just the difference between the Segment Tree rooted at $root_r$ and the Segment Tree rooted at $root_{l-1}$, i.e. every vertex in the $[l \dots r]$ Segment Tree can be computed with the vertex of the $root_r$ tree minus the vertex of the $root_{l-1}$ tree.

In the implementation of the `find_kth` function this can be handled by passing two vertex pointer and computing the count/sum of the current segment as difference of the two counts/sums of the vertices.

Here are the modified `build`, `update` and `find_kth` functions

```
Vertex* build(int tl, int tr) {
    if (tl == tr)
        return new Vertex(0);
    int tm = (tl + tr) / 2;
    return new Vertex(build(tl, tm), build(tm+1, tr));
}

Vertex* update(Vertex* v, int tl, int tr, int pos) {
    if (tl == tr)
        return new Vertex(v->sum+1);
    int tm = (tl + tr) / 2;
    if (pos <= tm)
        return new Vertex(update(v->l, tl, tm, pos), v->r);
    else
        return new Vertex(v->l, update(v->r, tm+1, tr, pos));
}

int find_kth(Vertex* vl, Vertex *vr, int tl, int tr, int k) {
    if (tl == tr)
        return tl;
    int tm = (tl + tr) / 2, left_count = vr->l->sum - vl->l->sum;
    if (left_count >= k)
        return find_kth(vl->l, vr->l, tl, tm, k);
    return find_kth(vl->r, vr->r, tm+1, tr, k-left_count);
}
```

As already written above, we need to store the root of the initial Segment Tree, and also all the roots after each update. Here is the code for building a persistent Segment Tree over an vector `a` with elements in the range `[0, MAX_VALUE]`.

```
int tl = 0, tr = MAX_VALUE + 1;
std::vector<Vertex*> roots;
roots.push_back(build(tl, tr));
for (int i = 0; i < a.size(); i++) {
    roots.push_back(update(roots.back(), tl, tr, a[i]));
}

// find the 5th smallest number from the subarray [a[2], a[3], ..., a[19]]
int result = find_kth(roots[2], roots[20], tl, tr, 5);
```

Now to the restrictions on the array elements: We can actually transform any array to such an array by index compression. The smallest element in the array will get assigned the value 0, the second smallest the value 1, and so forth. It is easy to generate lookup tables (e.g. using `map`), that convert a value to its index and vice versa in $O(\log n)$ time.

Dynamic segment tree

(Called so because its shape is dynamic and the nodes are usually dynamically allocated. Also known as *implicit segment tree* or *sparse segment tree*.)

Previously, we considered cases when we have the ability to build the original segment tree. But what to do if the original size is filled with some default element, but its size does not allow you to completely build up to it in advance?

We can solve this problem by creating a segment tree lazily (incrementally). Initially, we will create only the root, and we will create the other vertices only when we need them. In this case, we will use the implementation on pointers (before going to the vertex children, check whether they are created, and if not, create them). Each query has still only the complexity $O(\log n)$, which is small enough for most use-cases (e.g. $\log_2 10^9 \approx 30$).

In this implementation we have two queries, adding a value to a position (initially all values are 0), and computing the sum of all values in a range. `Vertex(0, n)` will be the root vertex of the implicit tree.

```
struct Vertex {
    int left, right;
    int sum = 0;
    Vertex *left_child = nullptr, *right_child = nullptr;

    Vertex(int lb, int rb) {
        left = lb;
        right = rb;
    }

    void extend() {
        if (!left_child && left + 1 < right) {
            int t = (left + right) / 2;
            left_child = new Vertex(left, t);
            right_child = new Vertex(t, right);
        }
    }

    void add(int k, int x) {
        extend();
        sum += x;
        if (left_child) {
            if (k < left_child->right)
                left_child->add(k, x);
            else
                right_child->add(k, x);
        }
    }

    int get_sum(int lq, int rq) {
        if (lq <= left && right <= rq)
            return sum;
        if (max(left, lq) >= min(right, rq))
            return 0;
        extend();
        return left_child->get_sum(lq, rq) + right_child->get_sum(lq, rq);
    }
};
```

Obviously this idea can be extended in lots of different ways. E.g. by adding support for range updates via lazy propagation.

Practice Problems

- [SPOJ - KQUERY](#) [Persistent segment tree / Merge sort tree]
- [Codeforces - Xenia and Bit Operations](#)
- [UVA 11402 - Ahoy, Pirates!](#)
- [SPOJ - GSS3](#)
- [Codeforces - Distinct Characters Queries](#)

- [Codeforces - Knight Tournament](#) [For beginners]
- [Codeforces - Ant colony](#)
- [Codeforces - Brazil and Park](#)
- [Codeforces - Circular RMQ](#)
- [Codeforces - Lucky Array](#)
- [Codeforces - The Child and Sequence](#)
- [Codeforces - DZY Loves Fibonacci Numbers](#) [Lazy propagation]
- [Codeforces - Alphabet Permutations](#)
- [Codeforces - Eyes Closed](#)
- [Codeforces - Kefa and Watch](#)
- [Codeforces - A Simple Task](#)
- [Codeforces - SUM and REPLACE](#)
- [Codeforces - XOR on Segment](#) [Lazy propagation]
- [Codeforces - Please, another Queries on Array?](#) [Lazy propagation]
- [COCI - Deda](#) [Last element smaller or equal to x / Binary search]
- [Codeforces - The Untended Antiquity](#) [2D]
- [CSES - Hotel Queries](#)
- [CSES - Polynomial Queries](#)
- [CSES - Range Updates and Sums](#)

Contributors:

[Jakob Kogler](#) (76.02%)
 [KhodorVan](#) (5.5%)
 [arjan-bal](#) (4.58%)
 [Lorenzo Rossi](#) (1.83%)
 [Gaurang Tandon](#) (1.67%)
 [Hidayat Khan](#) (1.33%)
 [jxu](#) (1.25%)
[Anastasios Koutian](#) (0.92%)
[Your name](#) (0.92%)
[Oleksandr Kulkov](#) (0.58%)
[Kerollos Magdy](#) (0.5%)
[L1nkus](#) (0.5%)
[Prashant Tripathi](#) (0.5%)
[Linkus000](#) (0.42%)
[Michael Hayter](#) (0.42%)
[Wiktor Kuchta](#) (0.42%)
[wiktorkuchta](#) (0.42%)
[huggin](#) (0.33%)
[Rupanshu Yadav](#) (0.25%)
[SiddharthEEE](#) (0.17%)
[Taichi Yamakawa](#) (0.17%)
[Abhishek-Saini](#) (0.08%)
[Anmol Gautam](#) (0.08%)
[ArthurConmy](#) (0.08%)
[Aryaman](#) (0.08%)
[ChangMarkusYu](#) (0.08%)
[Dev Choganwala](#) (0.08%)
[Heng](#) (0.08%)
[Jatin Grover](#) (0.08%)
[Mehrdad](#) (0.08%)
[Michael Zegar](#) (0.08%)
[Naman Bhalla](#) (0.08%)
[Rodion Gorkovenko](#) (0.08%)
[Sanskar Santosh Totla](#) (0.08%)
[Tanmay Sinha](#) (0.08%)
[boxlesscat](#) (0.08%)
[idleft](#) (0.08%)

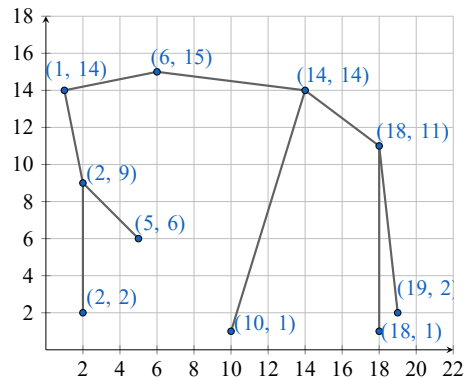
Translated From: e-maxx.ru

Treap (Cartesian tree)

A treap is a data structure which combines binary tree and binary heap (hence the name: tree + heap $\Rightarrow$ Treap).

More specifically, treap is a data structure that stores pairs (X, Y) in a binary tree in such a way that it is a binary search tree by X and a binary heap by Y . If some node of the tree contains values (X_0, Y_0) , all nodes in the left subtree have $X \leq X_0$, all nodes in the right subtree have $X_0 \leq X$, and all nodes in both left and right subtrees have $Y \leq Y_0$.

A treap is also often referred to as a "cartesian tree", as it is easy to embed it in a Cartesian plane:



Treaps have been proposed by Raimund Siedel and Cecilia Aragon in 1989.

Advantages of such data organisation

In such implementation, X values are the keys (and at same time the values stored in the treap), and Y values are called **priorities**. Without priorities, the treap would be a regular binary search tree by X , and one set of X values could correspond to a lot of different trees, some of them degenerate (for example, in the form of a linked list), and therefore extremely slow (the main operations would have $O(N)$ complexity).

At the same time, **priorities** (when they're unique) allow to **uniquely** specify the tree that will be constructed (of course, it does not depend on the order in which values are added), which can be proven using corresponding theorem. Obviously, if you **choose the priorities randomly**, you will get non-degenerate trees on average, which will ensure $O(\log N)$ complexity for the main operations. Hence another name of this data structure - **randomized binary search tree**.

Operations

A treap provides the following operations:

- **Insert (X, Y)** in $O(\log N)$.
Adds a new node to the tree. One possible variant is to pass only X and generate Y randomly inside the operation.
- **Search (X)** in $O(\log N)$.
Looks for a node with the specified key value X . The implementation is the same as for an ordinary binary search tree.
- **Erase (X)** in $O(\log N)$.
Looks for a node with the specified key value X and removes it from the tree.
- **Build $(X_1, \dots, X_N)$** in $O(N)$.
Builds a tree from a list of values. This can be done in linear time (assuming that $X_1, \dots, X_N$ are sorted).
- **Union (T_1, T_2)** in $O(M \log(N/M))$.
Merges two trees, assuming that all the elements are different. It is possible to achieve the same complexity if duplicate elements should be

removed during merge.

- **Intersect** (T_1, T_2) in $O(M \log(N/M))$.

Finds the intersection of two trees (i.e. their common elements). We will not consider the implementation of this operation here.

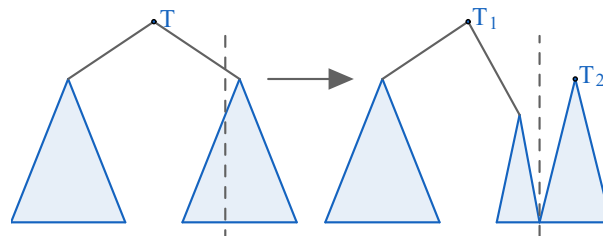
In addition, due to the fact that a treap is a binary search tree, it can implement other operations, such as finding the K -th largest element or finding the index of an element.

Implementation Description

In terms of implementation, each node contains X , Y and pointers to the left (L) and right (R) children.

We will implement all the required operations using just two auxiliary operations: Split and Merge.

Split



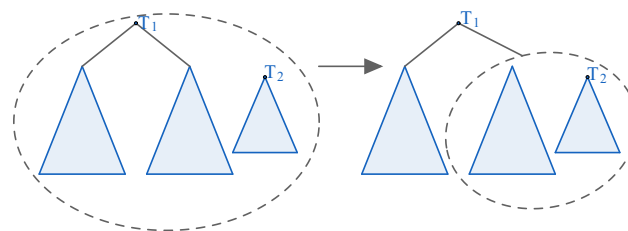
Split (T, X) separates tree T in 2 subtrees L and R trees (which are the return values of split) so that L contains all elements with key $X_L \leq X$, and R contains all elements with key $X_R > X$. This operation has $O(\log N)$ complexity and is implemented using a clean recursion:

1. If the value of the root node (R) is $\leq X$, then L would at least consist of $R \rightarrow L$ and R . We then call split on $R \rightarrow R$, and note its split result as L' and R' . Finally, L would also contain L' , whereas $R = R'$.
2. If the value of the root node (R) is $> X$, then R would at least consist of R and $R \rightarrow R$. We then call split on $R \rightarrow L$, and note its split result as L' and R' . Finally, $L = L'$, whereas R would also contain R' .

Thus, the split algorithm is:

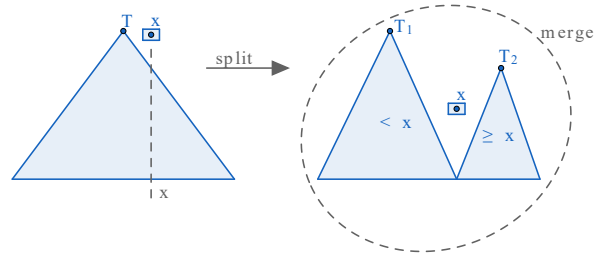
1. decide which subtree the root node would belong to (left or right)
2. recursively call split on one of its children
3. create the final result by reusing the recursive split call.

Merge



Merge (T_1, T_2) combines two subtrees T_1 and T_2 and returns the new tree. This operation also has $O(\log N)$ complexity. It works under the assumption that T_1 and T_2 are ordered (all keys X in T_1 are smaller than keys in T_2). Thus, we need to combine these trees without violating the order of priorities Y . To do this, we choose as the root the tree which has higher priority Y in the root node, and recursively call Merge for the other tree and the corresponding subtree of the selected root node.

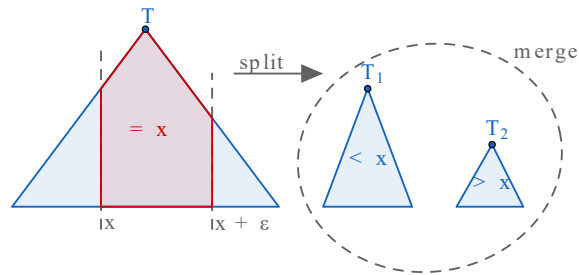
Insert



Now implementation of **Insert** (X, Y) becomes obvious. First we descend in the tree (as in a regular binary search tree by X), and stop at the first node in which the priority value is less than Y . We have found the place where we will insert the new element. Next, we call **Split** (T, X) on the subtree starting at the found node, and use returned subtrees L and R as left and right children of the new node.

Alternatively, insert can be done by splitting the initial treap on X and doing 2 merges with the new node (see the picture).

Erase



Implementation of **Erase** (X) is also clear. First we descend in the tree (as in a regular binary search tree by X), looking for the element we want to delete. Once the node is found, we call **Merge** on its children and put the return value of the operation in the place of the element we're deleting.

Alternatively, we can factor out the subtree holding X with 2 split operations and merge the remaining treaps (see the picture).

Build

We implement **Build** operation with $O(N \log N)$ complexity using N **Insert** calls.

Union

Union (T_1, T_2) has theoretical complexity $O(M \log(N/M))$, but in practice it works very well, probably with a very small hidden constant. Let's assume without loss of generality that $T_1 \rightarrow Y > T_2 \rightarrow Y$, i. e. root of T_1 will be the root of the result. To get the result, we need to merge trees $T_1 \rightarrow L, T_1 \rightarrow R$ and T_2 in two trees which could be children of T_1 root. To do this, we call **Split** ($T_2, T_1 \rightarrow X$), thus splitting T_2 in two parts L and R , which we then recursively combine with children of T_1 : **Union** ($T_1 \rightarrow L, L$) and **Union** ($T_1 \rightarrow R, R$), thus getting left and right subtrees of the result.

Implementation

```
struct item {
    int key, prior;
    item *l, *r;
    item () { }
    item (int key) : key(key), prior(rand()), l(NULL), r(NULL) { }
    item (int key, int prior) : key(key), prior(prior), l(NULL), r(NULL) { }
};
typedef item* pitem;
```

This is our item definition. Note there are two child pointers, and an integer key (for the BST) and an integer priority (for the heap). The priority is assigned using a random number generator.

```
void split (pitem t, int key, pitem & l, pitem & r) {
    if (!t)
        l = r = NULL;
    else if (t->key <= key)
        split (t->r, key, t->r, r), l = t;
    else
        split (t->l, key, l, t->l), r = t;
}
```

`t` is the treap to split, and `key` is the BST value by which to split. Note that we do not `return` the result values anywhere, instead, we just use them like so:

```
pitem l = nullptr, r = nullptr;
split(t, 5, l, r);
if (l) cout << "Left subtree size: " << (l->size) << endl;
if (r) cout << "Right subtree size: " << (r->size) << endl;
```

This `split` function can be tricky to understand, as it has both pointers (`pitem`) as well as reference to those pointers (`pitem &l`). Let us understand in words what the function call `split(t, k, l, r)` intends: "split treap `t` by value `k` into two treaps, and store the left treaps in `l` and right treap in `r`". Great! Now, let us apply this definition to the two recursive calls, using the case work we analyzed in the previous section: (The first if condition is a trivial base case for an empty treap)

1. When the root node value is $\leq$ `key`, we call `split (t->r, key, t->r, r)`, which means: "split treap `t->r` (right subtree of `t`) by value `key` and store the left subtree in `t->r` and right subtree in `r`". After that, we set `l = t`. Note now that the `l` result value contains `t->l`, `t` as well as `t->r` (which is the result of the recursive call we made) all already merged in the correct order! You should pause to ensure that this result of `l` and `r` corresponds exactly with what we discussed earlier in Implementation Description.
2. When the root node value is greater than `key`, we call `split (t->l, key, l, t->l)`, which means: "split treap `t->l` (left subtree of `t`) by value `key` and store the left subtree in `l` and right subtree in `t->l`". After that, we set `r = t`. Note now that the `r` result value contains `t->l` (which is the result of the recursive call we made), `t` as well as `t->r`, all already merged in the correct order! You should pause to ensure that this result of `l` and `r` corresponds exactly with what we discussed earlier in Implementation Description.

If you're still having trouble understanding the implementation, you should look at it *inductively*, that is: do *not* try to break down the recursive calls over and over again. Assume the split implementation works correct on empty treap, then try to run it for a single node treap, then a two node treap, and so on, each time reusing your knowledge that split on smaller treaps works.

```
void insert (pitem & t, pitem it) {
    if (!t)
        t = it;
    else if (it->prior > t->prior)
        split (t, it->key, it->l, it->r), t = it;
    else
        insert (t->key <= it->key ? t->r : t->l, it);
}

void merge (pitem & t, pitem l, pitem r) {
    if (!l || !r)
        t = l ? l : r;
    else if (l->prior > r->prior)
        merge (l->r, l->r, r), t = l;
    else
        merge (r->l, l, r->l), t = r;
}

void erase (pitem & t, int key) {
    if (t->key == key) {
        pitem th = t;
        merge (t, t->l, t->r);
        delete th;
    }
    else
        erase (key < t->key ? t->l : t->r, key);
}

pitem unite (pitem l, pitem r) {
    if (!l || !r) return l ? l : r;
    if (l->prior < r->prior) swap (l, r);
    pitem lt, rt;
    split (r, l->key, lt, rt);
}
```

```

l->l = unite (l->l, lt);
l->r = unite (l->r, rt);
return l;
}

```

Maintaining the sizes of subtrees

To extend the functionality of the treap, it is often necessary to store the number of nodes in subtree of each node - field `int cnt` in the `item` structure. For example, it can be used to find K-th largest element of tree in $O(\log N)$, or to find the index of the element in the sorted list with the same complexity. The implementation of these operations will be the same as for the regular binary search tree.

When a tree changes (nodes are added or removed etc.), `cnt` of some nodes should be updated accordingly. We'll create two functions: `cnt()` will return the current value of `cnt` or 0 if the node does not exist, and `upd_cnt()` will update the value of `cnt` for this node assuming that for its children L and R the values of `cnt` have already been updated. Evidently it's sufficient to add calls of `upd_cnt()` to the end of `insert`, `erase`, `split` and `merge` to keep `cnt` values up-to-date.

```

int cnt (pitem t) {
    return t ? t->cnt : 0;
}

void upd_cnt (pitem t) {
    if (t)
        t->cnt = 1 + cnt(t->l) + cnt (t->r);
}

```

Building a Treap in $O(N)$ in offline mode

Given a sorted list of keys, it is possible to construct a treap faster than by inserting the keys one at a time which takes $O(N \log N)$. Since the keys are sorted, a balanced binary search tree can be easily constructed in linear time. The heap values `Y` are initialized randomly and then can be heapified independent of the keys `X` to build the heap in $O(N)$.

```

void heapify (pitem t) {
    if (!t) return;
    pitem max = t;
    if (t->l != NULL && t->l->prior > max->prior)
        max = t->l;
    if (t->r != NULL && t->r->prior > max->prior)
        max = t->r;
    if (max != t) {
        swap (t->prior, max->prior);
        heapify (max);
    }
}

pitem build (int * a, int n) {
    // Construct a treap on values {a[0], a[1], ..., a[n - 1]}
    if (n == 0) return NULL;
    int mid = n / 2;
    pitem t = new item (a[mid], rand ());
    t->l = build (a, mid);
    t->r = build (a + mid + 1, n - mid - 1);
    heapify (t);
    upd_cnt(t);
    return t;
}

```

Note: calling `upd_cnt(t)` is only necessary if you need the subtree sizes.

The approach above always provides a perfectly balanced tree, which is generally good for practical purposes, but at the cost of not preserving the priorities that were initially assigned to each node. Thus, this approach is not feasible to solve the following problem:

acmsguru - Cartesian Tree

Given a sequence of pairs (x_i, y_i) , construct a cartesian tree on them. All x_i and all y_i are unique.

Note that in this problem priorities are not random, hence just inserting vertices one by one could provide a quadratic solution.

One of possible solutions here is to find for each element the closest elements to the left and to the right which have a smaller priority than this element. Among these two elements, the one with the larger priority must be the parent of the current element.

This problem is solvable with a [minimum stack](#) modification in linear time:

```
void connect(auto from, auto to) {
    vector<pitem> st;
    for(auto it: ranges::subrange(from, to)) {
        while(!st.empty() && st.back()->prior > it->prior) {
            st.pop_back();
        }
        if(!st.empty()) {
            if(!it->p || it->p->prior < st.back()->prior) {
                it->p = st.back();
            }
        }
        st.push_back(it);
    }
}

pitem build(int *x, int *y, int n) {
    vector<pitem> nodes(n);
    for(int i = 0; i < n; i++) {
        nodes[i] = new item(x[i], y[i]);
    }
    connect(nodes.begin(), nodes.end());
    connect(nodes.rbegin(), nodes.rend());
    for(int i = 0; i < n; i++) {
        if(nodes[i]->p) {
            if(nodes[i]->p->key < nodes[i]->key) {
                nodes[i]->p->r = nodes[i];
            } else {
                nodes[i]->p->l = nodes[i];
            }
        }
    }
    return nodes[min_element(y, y + n) - y];
}
```

Implicit Treaps

Implicit treap is a simple modification of the regular treap which is a very powerful data structure. In fact, implicit treap can be considered as an array with the following procedures implemented (all in $O(\log N)$ in the online mode):

- Inserting an element in the array in any location
- Removal of an arbitrary element
- Finding sum, minimum / maximum element etc. on an arbitrary interval
- Addition, painting on an arbitrary interval
- Reversing elements on an arbitrary interval

The idea is that the keys should be null-based **indices** of the elements in the array. But we will not store these values explicitly (otherwise, for example, inserting an element would cause changes of the key in $O(N)$ nodes of the tree).

Note that the key of a node is the number of nodes less than it (such nodes can be present not only in its left subtree but also in left subtrees of its ancestors). More specifically, the **implicit key** for some node T is the number of vertices $\text{cnt}(T \rightarrow L)$ in the left subtree of this node plus similar values $\text{cnt}(P \rightarrow L) + 1$ for each ancestor P of the node T , if T is in the right subtree of P .

Now it's clear how to calculate the implicit key of current node quickly. Since in all operations we arrive to any node by descending in the tree, we can just accumulate this sum and pass it to the function. If we go to the left subtree, the accumulated sum does not change, if we go to the right subtree it increases by $\text{cnt}(T \rightarrow L) + 1$.

Here are the new implementations of **Split** and **Merge**:

```
void merge (pitem & t, pitem l, pitem r) {
    if (!l || !r)
        t = l ? l : r;
    else if (l->prior > r->prior)
        merge (l->r, l->r, r), t = l;
    else
```

```

    merge (r->l, l, r->l), t = r;
    upd_cnt (t);
}

void split (pitem t, pitem & l, pitem & r, int key, int add = 0) {
    if (!t)
        return void( l = r = 0 );
    int cur_key = add + cnt(t->l); //implicit key
    if (key <= cur_key)
        split (t->l, l, t->l, key, add), r = t;
    else
        split (t->r, t->r, r, key, add + 1 + cnt(t->l)), l = t;
    upd_cnt (t);
}

```

In the implementation above, after the call of `split(T, T1, T2, k)`, the tree T_1 will consist of first k elements of T (that is, of elements having their implicit key less than k) and T_2 will consist of all the rest.

Now let's consider the implementation of various operations on implicit treaps:

- **Insert element.**

Suppose we need to insert an element at position `pos`. We divide the treap into two parts, which correspond to arrays `[0..pos - 1]` and `[pos..sz]`; to do this we call `split(T, T1, T2, pos)`. Then we can combine tree T_1 with the new vertex by calling `merge(T1, T1, new item)` (it is easy to see that all preconditions are met). Finally, we combine trees T_1 and T_2 back into T by calling `merge(T, T1, T2)`.

- **Delete element.**

This operation is even easier: find the element to be deleted T , perform merge of its children L and R , and replace the element T with the result of merge. In fact, element deletion in the implicit treap is exactly the same as in the regular treap.

- Find **sum / minimum**, etc. on the interval.

First, create an additional field `F` in the `item` structure to store the value of the target function for this node's subtree. This field is easy to maintain similarly to maintaining sizes of subtrees: create a function which calculates this value for a node based on values for its children and add calls of this function in the end of all functions which modify the tree.

Second, we need to know how to process a query for an arbitrary interval `[A; B]`.

To get a part of tree which corresponds to the interval `[A; B]`, we need to call `split(T, T2, T3, B + 1)`, and then `split(T2, T1, T2, A)`: after this T_2 will consist of all the elements in the interval `[A; B]`, and only of them. Therefore, the response to the query will be stored in the field `F` of the root of T_2 . After the query is answered, the tree has to be restored by calling `merge(T, T1, T2)` and `merge(T, T, T3)`.

- **Addition / painting** on the interval.

We act similarly to the previous paragraph, but instead of the field `F` we will store a field `add` which will contain the added value for the subtree (or the value to which the subtree is painted). Before performing any operation we have to "push" this value correctly - i.e. change $T \rightarrow L \rightarrow \text{add}$ and $T \rightarrow R \rightarrow \text{add}$, and to clean up `add` in the parent node. This way after any changes to the tree the information will not be lost.

- **Reverse** on the interval.

This is again similar to the previous operation: we have to add boolean flag `rev` and set it to true when the subtree of the current node has to be reversed. "Pushing" this value is a bit complicated - we swap children of this node and set this flag to true for them.

Here is an example implementation of the implicit treap with reverse on the interval. For each node we store field called `value` which is the actual value of the array element at current position. We also provide implementation of the function `output()`, which outputs an array that corresponds to the current state of the implicit treap.

```

typedef struct item * pitem;
struct item {
    int prior, value, cnt;
    bool rev;
    pitem l, r;
};

int cnt (pitem it) {
    return it ? it->cnt : 0;
}

void upd_cnt (pitem it) {
    if (it)
        it->cnt = cnt(it->l) + cnt(it->r) + 1;
}

```

```

void push (pitem it) {
    if (it && it->rev) {
        it->rev = false;
        swap (it->l, it->r);
        if (it->l) it->l->rev ^= true;
        if (it->r) it->r->rev ^= true;
    }
}

void merge (pitem & t, pitem l, pitem r) {
    push (l);
    push (r);
    if (!l || !r)
        t = l ? l : r;
    else if (l->prior > r->prior)
        merge (l->r, l->r, r), t = l;
    else
        merge (r->l, l, r->l), t = r;
    upd_cnt (t);
}

void split (pitem t, pitem & l, pitem & r, int key, int add = 0) {
    if (!t)
        return void( l = r = 0 );
    push (t);
    int cur_key = add + cnt(t->l);
    if (key <= cur_key)
        split (t->l, l, t->l, key, add), r = t;
    else
        split (t->r, t->r, r, key, add + 1 + cnt(t->l)), l = t;
    upd_cnt (t);
}

void reverse (pitem t, int l, int r) {
    pitem t1, t2, t3;
    split (t, t1, t2, l);
    split (t2, t2, t3, r-1+1);
    t2->rev ^= true;
    merge (t, t1, t2);
    merge (t, t, t3);
}

void output (pitem t) {
    if (!t) return;
    push (t);
    output (t->l);
    printf ("%d ", t->value);
    output (t->r);
}

```

Literature

- [Blelloch, Reid-Miller "Fast Set Operations Using Treaps"](#)

Practice Problems

- [SPOJ - Ada and Aphids](#)
- [SPOJ - Ada and Harvest](#)
- [Codeforces - Radio Stations](#)
- [SPOJ - Ghost Town](#)
- [SPOJ - Arrangement Validity](#)
- [SPOJ - All in One](#)
- [Codeforces - Dog Show](#)
- [Codeforces - Yet Another Array Queries Problem](#)
- [SPOJ - Mean of Array](#)
- [SPOJ - TWIST](#)
- [SPOJ - KOILINE](#)

- [CodeChef - The Prestige](#)
- [Codeforces - T-Shirts](#)
- [Codeforces - Wizards and Roads](#)
- [Codeforces - Yaroslav and Points](#)

Contributors:

[Sergii Lischuk](#) (41.84%) [Oleksandr Kulkov](#) (28.05%) [Mariia Mykhailova](#) (8.97%) [Gaurang Tandon](#) (7.36%) [Soumik Sarkar](#) (6.44%) [Morass](#) (3.22%)
[Dalgerok](#) (1.38%) [Anton Algmyr](#) (0.92%) [Jakob Kogler](#) (0.92%) [princezna](#) (0.46%) [Max](#) (0.23%) [yousvf](#) (0.23%)

Original

Sqrt Tree

Given an array a that contains n elements and the operation $\circ$ that satisfies associative property: $(x \circ y) \circ z = x \circ (y \circ z)$ is true for any x, y, z .

So, such operations as $\gcd$, $\min$, $\max$, $+$, and , or , xor , etc. satisfy these conditions.

Also we have some queries $q(l, r)$. For each query, we need to compute $a_l \circ a_{l+1} \circ \dots \circ a_r$.

Sqrt Tree can process such queries in $O(1)$ time with $O(n \cdot \log \log n)$ preprocessing time and $O(n \cdot \log \log n)$ memory.

Description

Building sqrt decomposition

Let's make a [sqrt decomposition](#). We divide our array in $\sqrt{n}$ blocks, each block has size $\sqrt{n}$. For each block, we compute:

1. Answers to the queries that lie in the block and begin at the beginning of the block (prefixOp)
2. Answers to the queries that lie in the block and end at the end of the block (suffixOp)

And we'll compute an additional array:

1. between $_{i,j}$ (for $i \leq j$) - answer to the query that begins at the start of block i and ends at the end of block j . Note that we have $\sqrt{n}$ blocks, so the size of this array will be $O(\sqrt{n}^2) = O(n)$.

Let's see the example.

Let $\circ$ be $+$ (we calculate sum on a segment) and we have the following array a :

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

It will be divided onto three blocks: $\{1, 2, 3\}$, $\{4, 5, 6\}$ and $\{7, 8, 9\}$.

For first block prefixOp is $\{1, 3, 6\}$ and suffixOp is $\{6, 5, 3\}$.

For second block prefixOp is $\{4, 9, 15\}$ and suffixOp is $\{15, 11, 6\}$.

For third block prefixOp is $\{7, 15, 24\}$ and suffixOp is $\{24, 17, 9\}$.

between array is:

```
{
  {6, 21, 45},
  {0, 15, 39},
  {0, 0, 24}
}
```

(we assume that invalid elements where $i > j$ are filled with zeroes)

It's obvious to see that these arrays can be easily calculated in $O(n)$ time and memory.

We already can answer some queries using these arrays. If the query doesn't fit into one block, we can divide it into three parts: suffix of a block, then some segment of contiguous blocks and then prefix of some block. We can answer a query by dividing it into three parts and taking our operation of some value from suffixOp, then some value from between, then some value from prefixOp.

But if we have queries that entirely fit into one block, we cannot process them using these three arrays. So, we need to do something.

Making a tree

We cannot answer only the queries that entirely fit in one block. But what if **we build the same structure as described above for each block**? Yes, we can do it. And we do it recursively, until we reach the block size of 1 or 2. Answers for such blocks can be calculated easily in $O(1)$.

So, we get a tree. Each node of the tree represents some segment of the array. Node that represents array segment with size k has $\sqrt{k}$ children – for each block. Also each node contains the three arrays described above for the segment it contains. The root of the tree represents the entire array. Nodes with segment lengths 1 or 2 are leaves.

Also it's obvious that the height of this tree is $O(\log \log n)$, because if some vertex of the tree represents an array with length k , then its children have length $\sqrt{k}$. $\log(\sqrt{k}) = \frac{\log k}{2}$, so $\log k$ decreases two times every layer of the tree and so its height is $O(\log \log n)$. The time for building and memory usage will be $O(n \cdot \log \log n)$, because every element of the array appears exactly once on each layer of the tree.

Now we can answer the queries in $O(\log \log n)$. We can go down on the tree until we meet a segment with length 1 or 2 (answer for it can be calculated in $O(1)$ time) or meet the first segment in which our query doesn't fit entirely into one block. See the first section on how to answer the query in this case.

OK, now we can do $O(\log \log n)$ per query. Can it be done faster?

Optimizing the query complexity

One of the most obvious optimization is to binary search the tree node we need. Using binary search, we can reach the $O(\log \log \log n)$ complexity per query. Can we do it even faster?

The answer is yes. Let's assume the following two things:

1. Each block size is a power of two.
2. All the blocks are equal on each layer.

To reach this, we can add some zero elements to our array so that its size becomes a power of two.

When we use this, some block sizes may become twice larger to be a power of two, but it still be $O(\sqrt{k})$ in size and we keep linear complexity for building the arrays in a segment.

Now, we can easily check if the query fits entirely into a block with size 2^k . Let's write the ranges of the query, l and r (we use 0-indexation) in binary form. For instance, let's assume $k = 4$, $l = 39$, $r = 46$. The binary representation of l and r is:

$$l = 39_{10} = 100111_2$$

$$r = 46_{10} = 101110_2$$

Remember that one layer contains segments of the equal size, and the block on one layer have also equal size (in our case, their size is $2^k = 2^4 = 16$). The blocks cover the array entirely, so the first block covers elements $(0 - 15)$ ($(000000_2 - 001111_2)$ in binary), the second one covers elements $(16 - 31)$ ($(010000_2 - 011111_2)$ in binary) and so on. We see that the indices of the positions covered by one block may differ only in k (in our case, 4) last bits. In our case l and r have equal bits except four lowest, so they lie in one block.

So, we need to check if nothing more that k smallest bits differ (or $l \text{ xor } r$ doesn't exceed $2^k - 1$).

Using this observation, we can find a layer that is suitable to answer the query quickly. How to do this:

1. For each i that doesn't exceed the array size, we find the highest bit that is equal to 1. To do this quickly, we use DP and a precalculated array.
2. Now, for each $q(l, r)$ we find the highest bit of $l \text{ xor } r$ and, using this information, it's easy to choose the layer on which we can process the query easily. We can also use a precalculated array here.

For more details, see the code below.

So, using this, we can answer the queries in $O(1)$ each. Hooray! :)

Updating elements

We can also update elements in Sqrt Tree. Both single element updates and updates on a segment are supported.

Updating a single element

Consider a query `update(x, val)` that does the assignment $a_x = val$. We need to perform this query fast enough.

Naive approach

First, let's take a look of what is changed in the tree when a single element changes. Consider a tree node with length l and its arrays: `prefixOp`, `suffixOp` and `between`. It is easy to see that only $O(\sqrt{l})$ elements from `prefixOp` and `suffixOp` change (only inside the block with the changed element). $O(l)$ elements are changed in `between`. Therefore, $O(l)$ elements in the tree node are updated.

We remember that any element x is present in exactly one tree node at each layer. Root node (layer 0) has length $O(n)$, nodes on layer 1 have length $O(\sqrt{n})$, nodes on layer 2 have length $O(\sqrt{\sqrt{n}})$, etc. So the time complexity per update is $O(n + \sqrt{n} + \sqrt{\sqrt{n}} + \dots) = O(n)$.

But it's too slow. Can it be done faster?

An sqrt-tree inside the sqrt-tree

Note that the bottleneck of updating is rebuilding `between` of the root node. To optimize the tree, let's get rid of this array! Instead of `between` array, we store another sqrt-tree for the root node. Let's call it `index`. It plays the same role as `between` — answers the queries on segments of blocks. Note that the rest of the tree nodes don't have `index`, they keep their `between` arrays.

A sqrt-tree is *indexed*, if its root node has `index`. A sqrt-tree with `between` array in its root node is *unindexed*. Note that `index` is *unindexed* itself.

So, we have the following algorithm for updating an *indexed* tree:

- Update `prefixOp` and `suffixOp` in $O(\sqrt{n})$.
- Update `index`. It has length $O(\sqrt{n})$ and we need to update only one item in it (that represents the changed block). So, the time complexity for this step is $O(\sqrt{n})$. We can use the algorithm described in the beginning of this section (the "slow" one) to do it.
- Go into the child node that represents the changed block and update it in $O(\sqrt{n})$ with the "slow" algorithm.

Note that the query complexity is still $O(1)$: we need to use `index` in query no more than once, and this will take $O(1)$ time.

So, total time complexity for updating a single element is $O(\sqrt{n})$. Hooray! :)

Updating a segment

Sqrt-tree also can do things like assigning an element on a segment. `massUpdate(x, l, r)` means $a_i = x$ for all $l \leq i \leq r$.

There are two approaches to do this: one of them does `massUpdate` in $O(\sqrt{n} \cdot \log \log n)$, keeping $O(1)$ per query. The second one does `massUpdate` in $O(\sqrt{n})$, but the query complexity becomes $O(\log \log n)$.

We will do lazy propagation in the same way as it is done in segment trees: we mark some nodes as *lazy*, meaning that we'll push them when it's necessary. But one thing is different from segment trees: pushing a node is expensive, so it cannot be done in queries. On the layer 0, pushing a node takes $O(\sqrt{n})$ time. So, we don't push nodes inside queries, we only look if the current node or its parent are *lazy*, and just take it into account while performing queries.

First approach

In the first approach, we say that only nodes on layer 1 (with length $O(\sqrt{n})$) can be *lazy*. When pushing such node, it updates all its subtree including itself in $O(\sqrt{n} \cdot \log \log n)$. The `massUpdate` process is done as follows:

- Consider the nodes on layer 1 and blocks corresponding to them.
- Some blocks are entirely covered by `massUpdate`. Mark them as *lazy* in $O(\sqrt{n})$.
- Some blocks are partially covered. Note there are no more than two blocks of this kind. Rebuild them in $O(\sqrt{n} \cdot \log \log n)$. If they were *lazy*, take it into account.
- Update `prefixOp` and `suffixOp` for partially covered blocks in $O(\sqrt{n})$ (because there are only two such blocks).

- Rebuild the index in $O(\sqrt{n} \cdot \log \log n)$.

So we can do `massUpdate` fast. But how lazy propagation affects queries? They will have the following modifications:

- If our query entirely lies in a *lazy* block, calculate it and take *lazy* into account. $O(1)$.
- If our query consists of many blocks, some of which are *lazy*, we need to take care of *lazy* only on the leftmost and the rightmost block. The rest of the blocks are calculated using `index`, which already knows the answer on *lazy* block (because it's rebuilt after each modification). $O(1)$.

The query complexity still remains $O(1)$.

Second approach

In this approach, each node can be *lazy* (except root). Even nodes in `index` can be *lazy*. So, while processing a query, we have to look for *lazy* tags in all the parent nodes, i. e. query complexity will be $O(\log \log n)$.

But `massUpdate` becomes faster. It looks in the following way:

- Some blocks are fully covered with `massUpdate`. So, *lazy* tags are added to them. It is $O(\sqrt{n})$.
- Update `prefixOp` and `suffixOp` for partially covered blocks in $O(\sqrt{n})$ (because there are only two such blocks).
- Do not forget to update the index. It is $O(\sqrt{n})$ (we use the same `massUpdate` algorithm).
- Update `between` array for *unindexed* subtrees.
- Go into the nodes representing partially covered blocks and call `massUpdate` recursively.

Note that when we do the recursive call, we do prefix or suffix `massUpdate`. But for prefix and suffix updates we can have no more than one partially covered child. So, we visit one node on layer 1, two nodes on layer 2 and two nodes on any deeper level. So, the time complexity is $O(\sqrt{n} + \sqrt{\sqrt{n}} + \dots) = O(\sqrt{n})$. The approach here is similar to the segment tree mass update.

Implementation

The following implementation of Sqrt Tree can perform the following operations: build in $O(n \cdot \log \log n)$, answer queries in $O(1)$ and update an element in $O(\sqrt{n})$.

```
SqrtTreeItem op(const SqrtTreeItem &a, const SqrtTreeItem &b);

inline int log2Up(int n) {
    int res = 0;
    while ((1 << res) < n) {
        res++;
    }
    return res;
}

class SqrtTree {
private:
    int n, lg, indexSz;
    vector<SqrtTreeItem> v;
    vector<int> clz, layers, onLayer;
    vector< vector<SqrtTreeItem> > pref, suf, between;

    inline void buildBlock(int layer, int l, int r) {
        pref[layer][l] = v[l];
        for (int i = l+1; i < r; i++) {
            pref[layer][i] = op(pref[layer][i-1], v[i]);
        }
        suf[layer][r-1] = v[r-1];
        for (int i = r-2; i >= l; i--) {
            suf[layer][i] = op(v[i], suf[layer][i+1]);
        }
    }

    inline void buildBetween(int layer, int lBound, int rBound, int betweenOffs) {
        int bSzLog = (layers[layer]+1) >> 1;

```

```

int bCntLog = layers[layer] >> 1;
int bSz = 1 << bSzLog;
int bCnt = (rBound - lBound + bSz - 1) >> bSzLog;
for (int i = 0; i < bCnt; i++) {
    SqrtTreeItem ans;
    for (int j = i; j < bCnt; j++) {
        SqrtTreeItem add = suf[layer][lBound + (j << bSzLog)];
        ans = (i == j) ? add : op(ans, add);
        between[layer-1][betweenOffs + lBound + (i << bCntLog) + j] = ans;
    }
}

}

inline void buildBetweenZero() {
    int bSzLog = (lg+1) >> 1;
    for (int i = 0; i < indexSz; i++) {
        v[n+i] = suf[0][i << bSzLog];
    }
    build(1, n, n + indexSz, (1 << lg) - n);
}

inline void updateBetweenZero(int bid) {
    int bSzLog = (lg+1) >> 1;
    v[n+bid] = suf[0][bid << bSzLog];
    update(1, n, n + indexSz, (1 << lg) - n, n+bid);
}

void build(int layer, int lBound, int rBound, int betweenOffs) {
    if (layer >= (int)layers.size()) {
        return;
    }
    int bSz = 1 << ((layers[layer]+1) >> 1);
    for (int l = lBound; l < rBound; l += bSz) {
        int r = min(l + bSz, rBound);
        buildBlock(layer, l, r);
        build(layer+1, l, r, betweenOffs);
    }
    if (layer == 0) {
        buildBetweenZero();
    } else {
        buildBetween(layer, lBound, rBound, betweenOffs);
    }
}

void update(int layer, int lBound, int rBound, int betweenOffs, int x) {
    if (layer >= (int)layers.size()) {
        return;
    }
    int bSzLog = (layers[layer]+1) >> 1;
    int bSz = 1 << bSzLog;
    int blockIdx = (x - lBound) >> bSzLog;
    int l = lBound + (blockIdx << bSzLog);
    int r = min(l + bSz, rBound);
    buildBlock(layer, l, r);
    if (layer == 0) {
        updateBetweenZero(blockIdx);
    } else {
        buildBetween(layer, lBound, rBound, betweenOffs);
    }
    update(layer+1, l, r, betweenOffs, x);
}

inline SqrtTreeItem query(int l, int r, int betweenOffs, int base) {
    if (l == r) {
        return v[l];
    }
    if (l + 1 == r) {
        return op(v[l], v[r]);
    }
    int layer = onLayer[clz[(l - base) ^ (r - base)]];
    int bSzLog = (layers[layer]+1) >> 1;
    int bCntLog = layers[layer] >> 1;
    int lBound = (((l - base) >> layers[layer]) << layers[layer]) + base;
    int lBlock = ((l - lBound) >> bSzLog) + 1;
    int rBlock = ((r - lBound) >> bSzLog) - 1;
    SqrtTreeItem ans = suf[layer][l];
    if (lBlock <= rBlock) {
        SqrtTreeItem add = (layer == 0) ? (
            query(n + lBlock, n + rBlock, (1 << lg) - n, n)
        ) : (

```

```

        between[layer-1][betweenOffs + lBound + (lBlock << bCntLog) + rBlock]
    );
    ans = op(ans, add);
}
ans = op(ans, pref[layer][r]);
return ans;
}
public:
    inline SqrtTreeItem query(int l, int r) {
        return query(l, r, 0, 0);
    }

    inline void update(int x, const SqrtTreeItem &item) {
        v[x] = item;
        update(0, 0, n, 0, x);
    }

    SqrtTree(const vector<SqrtTreeItem>& a)
        : n((int)a.size()), lg(log2Up(n)), v(a), clz(1 << lg), onLayer(lg+1) {
        clz[0] = 0;
        for (int i = 1; i < (int)clz.size(); i++) {
            clz[i] = clz[i >> 1] + 1;
        }
        int tlg = lg;
        while (tlg > 1) {
            onLayer[tlg] = (int)layers.size();
            layers.push_back(tlg);
            tlg = (tlg+1) >> 1;
        }
        for (int i = lg-1; i >= 0; i--) {
            onLayer[i] = max(onLayer[i], onLayer[i+1]);
        }
        int betweenLayers = max(0, (int)layers.size() - 1);
        int bSzLog = (lg+1) >> 1;
        int bSz = 1 << bSzLog;
        indexSz = (n + bSz - 1) >> bSzLog;
        v.resize(n + indexSz);
        pref.assign(layers.size(), vector<SqrtTreeItem>(n + indexSz));
        suf.assign(layers.size(), vector<SqrtTreeItem>(n + indexSz));
        between.assign(betweenLayers, vector<SqrtTreeItem>((1 << lg) + bSz));
        build(0, 0, n, 0);
    }
};

```

Problems

[CodeChef - SEGPROD](#)

Contributors:

[Alexander Kernozhitsky](#) (94.02%) [Anton Ålgmyr](#) (3.98%) [Oleksandr Kulkov](#) (1.42%) [Jakob Kogler](#) (0.28%) [wiktorkuchta](#) (0.28%)

Translated From: e-maxx.ru

Randomized Heap

A randomized heap is a heap that, through using randomization, allows to perform all operations in expected logarithmic time.

A **min heap** is a binary tree in which the value of each vertex is less than or equal to the values of its children. Thus the minimum of the tree is always in the root vertex.

A max heap can be defined in a similar way: by replacing less with greater.

The default operations of a heap are:

- Adding a value
- Extracting the minimum
- Removing the minimum
- Merging two heaps (without deleting duplicates)
- Removing an arbitrary element (if its position in the tree is known)

A randomized heap can perform all these operations in expected $O(\log n)$ time with a very simple implementation.

Data structure

We can immediately describe the structure of the binary heap:

```
struct Tree {
    int value;
    Tree * l = nullptr;
    Tree * r = nullptr;
};
```

In the vertex we store a value. In addition we have pointers to the left and right children, which are point to null if the corresponding child doesn't exist.

Operations

It is not difficult to see, that all operations can be reduced to a single one: **merging** two heaps into one. Indeed, adding a new value to the heap is equivalent to merging the heap with a heap consisting of a single vertex with that value. Finding a minimum doesn't require any operation at all - the minimum is simply the value at the root. Removing the minimum is equivalent to the result of merging the left and right children of the root vertex. And removing an arbitrary element is similar. We merge the children of the vertex and replace the vertex with the result of the merge.

So we actually only need to implement the operation of merging two heaps. All other operations are trivially reduced to this operation.

Let two heaps T_1 and T_2 be given. It is clear that the root of each of these heaps contains its minimum. So the root of the resulting heap will be the minimum of these two values. So we compare both values, and use the smaller one as the new root. Now we have to combine the children of the selected vertex with the remaining heap. For this we select one of the children, and merge it with the remaining heap. Thus we again have the operation of merging two heaps. Sooner or later this process will end (the number of such steps is limited by the sum of the heights of the two heaps)

To achieve logarithmic complexity on average, we need to specify a method for choosing one of the two children so that the average path length is logarithmic. It is not difficult to guess, that we will make this decision **randomly**. Thus the implementation of the merging operation is as follows:

```
Tree* merge(Tree* t1, Tree* t2) {
    if (!t1 || !t2)
```

```

    return t1 ? t1 : t2;
    if (t2->value < t1->value)
        swap(t1, t2);
    if (rand() & 1)
        swap(t1->l, t1->r);
    t1->l = merge(t1->l, t2);
    return t1;
}

```

Here first we check if one of the heaps is empty, then we don't need to perform any merge action at all. Otherwise we make the heap `t1` the one with the smaller value (by swapping `t1` and `t2` if necessary). We want to merge the left child of `t1` with `t2`, therefore we randomly swap the children of `t1`, and then perform the merge.

Complexity

We introduce the random variable $h(T)$ which will denote the **length of the random path** from the root to the leaf (the length in the number of edges). It is clear that the algorithm `merge` performs $O(h(T_1) + h(T_2))$ steps. Therefore to understand the complexity of the operations, we must look into the random variable $h(T)$.

Expected value

We assume that the expectation $h(T)$ can be estimated from above by the logarithm of the number of vertices in the heap:

$$Eh(T) \leq \log(n + 1)$$

This can be easily proven by induction. Let L and R be the left and the right subtrees of the root T , and n_L and n_R the number of vertices in them ($n = n_L + n_R + 1$).

The following shows the induction step:

$$\begin{aligned}
 Eh(T) &= 1 + \frac{Eh(L) + Eh(R)}{2} \leq 1 + \frac{\log(n_L + 1) \log(n_R + 1)}{2} \\
 &= 1 + \log \sqrt{(n_L + 1)(n_R + 1)} = \log 2 \sqrt{(n_L + 1)(n_R + 1)} \\
 &\leq \log \frac{2((n_L + 1) + (n_R + 1))}{2} = \log(n_L + n_R + 2) = \log(n + 1)
 \end{aligned}$$

Exceeding the expected value

Of course we are still not happy. The expected value of $h(T)$ doesn't say anything about the worst case. It is still possible that the paths from the root to the vertices is on average much greater than $\log(n + 1)$ for a specific tree.

Let us prove that exceeding the expected value is indeed very small:

$$P\{h(T) > (c + 1) \log n\} < \frac{1}{n^c}$$

for any positive constant c .

Here we denote by P the set of paths from the root of the heap to the leaves where the length exceeds $(c + 1) \log n$. Note that for any path p of length $|p|$ the probability that it will be chosen as random path is $2^{-|p|}$. Therefore we get:

$$P\{h(T) > (c + 1) \log n\} = \sum_{p \in P} 2^{-|p|} < \sum_{p \in P} 2^{-(c+1) \log n} = |P| n^{-(c+1)} \leq n^{-c}$$

Complexity of the algorithm

Thus the algorithm `merge`, and hence all other operations expressed with it, can be performed in $O(\log n)$ on average.

Moreover for any positive constant ϵ there is a positive constant c , such that the probability that the operation will require more than $c \log n$ steps is less than $n^{-\epsilon}$ (in some sense this describes the worst case behavior of the algorithm).

Contributors:

[Jakob Kogler](#) (93.8%) [Oleksandr Kulkov](#) (4.65%) [Bergur Snorrason](#) (1.55%)

Original

Deleting from a data structure in $O(T(n) \log n)$

Suppose you have a data structure which allows adding elements in **true** $O(T(n))$. This article will describe a technique that allows deletion in $O(T(n) \log n)$ offline.

Algorithm

Each element lives in the data structure for some segments of time between additions and deletions. Let's build a segment tree over the queries. Each segment when some element is alive splits into $O(\log n)$ nodes of the tree. Let's put each query when we want to know something about the structure into the corresponding leaf. Now to process all queries we will run a DFS on the segment tree. When entering the node we will add all the elements that are inside this node. Then we will go further to the children of this node or answer the queries (if the node is a leaf). When leaving the node, we must undo the additions. Note that if we change the structure in $O(T(n))$ we can roll back the changes in $O(T(n))$ by keeping a stack of changes. Note that rollbacks break amortized complexity.

Notes

The idea of creating a segment tree over segments when something is alive may be used not only for data structure problems. See some problems below.

Implementation

This implementation is for the [dynamic connectivity](#) problem. It can add edges, remove edges and count the number of connected components.

```
struct dsu_save {
    int v, rnkv, u, rnku;

    dsu_save() {}

    dsu_save(int _v, int _rnkv, int _u, int _rnku)
        : v(_v), rnkv(_rnkv), u(_u), rnku(_rnku) {}
};

struct dsu_with_rollbacks {
    vector<int> p, rnk;
    int comps;
    stack<dsu_save> op;

    dsu_with_rollbacks() {}

    dsu_with_rollbacks(int n) {
        p.resize(n);
        rnk.resize(n);
        for (int i = 0; i < n; i++) {
            p[i] = i;
            rnk[i] = 0;
        }
        comps = n;
    }

    int find_set(int v) {
        return (v == p[v]) ? v : find_set(p[v]);
    }

    bool unite(int v, int u) {
        v = find_set(v);
        u = find_set(u);
        if (v == u)
            return false;
        comps--;
        if (rnk[v] > rnk[u])
            swap(v, u);
```

```

        op.push(dsu_save(v, rnk[v], u, rnk[u]));
        p[v] = u;
        if (rnk[u] == rnk[v])
            rnk[u]++;
        return true;
    }

    void rollback() {
        if (op.empty())
            return;
        dsu_save x = op.top();
        op.pop();
        comps++;
        p[x.v] = x.v;
        rnk[x.v] = x.rnk;
        p[x.u] = x.u;
        rnk[x.u] = x.rnk;
    }
};

struct query {
    int v, u;
    bool united;
    query(int _v, int _u) : v(_v), u(_u) {}
};

struct QueryTree {
    vector<vector<query>> t;
    dsu_with_rollback dsu;
    int T;

    QueryTree() {}

    QueryTree(int _T, int n) : T(_T) {
        dsu = dsu_with_rollback(n);
        t.resize(4 * T + 4);
    }

    void add_to_tree(int v, int l, int r, int ul, int ur, query& q) {
        if (ul > ur)
            return;
        if (l == ul && r == ur) {
            t[v].push_back(q);
            return;
        }
        int mid = (l + r) / 2;
        add_to_tree(2 * v, l, mid, ul, min(ur, mid), q);
        add_to_tree(2 * v + 1, mid + 1, r, max(ul, mid + 1), ur, q);
    }

    void add_query(query q, int l, int r) {
        add_to_tree(1, 0, T - 1, l, r, q);
    }

    void dfs(int v, int l, int r, vector<int>& ans) {
        for (query& q : t[v]) {
            q.united = dsu.unite(q.v, q.u);
        }
        if (l == r)
            ans[l] = dsu.comps;
        else {
            int mid = (l + r) / 2;
            dfs(2 * v, l, mid, ans);
            dfs(2 * v + 1, mid + 1, r, ans);
        }
        for (query q : t[v]) {
            if (q.united)
                dsu.rollback();
        }
    }

    vector<int> solve() {
        vector<int> ans(T);
        dfs(1, 0, T - 1, ans);
        return ans;
    }
};

```

Problems

- [Codeforces - Connect and Disconnect](#)
- [Codeforces - Addition on Segments](#)
- [Codeforces - Extending Set of Points](#)

Contributors:

[Yury Semenov](#) (95.54%) [Jakob Kogler](#) (3.18%) [Oleksandr Kulkov](#) (1.27%)